ML01 期末复习 · 详细整理

Session 2 / 3 / 5 / 6 / 7 + Transformer Extra（201/202/203）

题面英文原文 + 中文翻译 + 英文作答 + 中文解析

ML01 期末复习 · 详细整理

> 格式：每个考点 = 文件索引/位置 → **英文原文（完整）+ 中文翻译** → **答案（English 作答 + 中文解析）**。
>
> 范围以「今日老师确认」为准：Session 1/4 不考；BCE、CE(含多分类 CE) 不考。
>
> 进度：✅ Session 2 ｜ 待续：Session 3、5、6、7、201-205。

Session 2　逻辑回归 Logistic Regression（重点章节）

确认考点：**① 逻辑回归代码(fit+sigmoid) ② sigmoid 画图+公式 ③ TF Playground 特征工程(2-ag) ④ 加深网络/两螺旋(2-ah)。**

不考：BCE / CE-vs-BCE、dw·db 的**推导**、predict 函数。softmax 见 Session 3。

考点 2.1　逻辑回归代码（重中之重：主考 fit）

文件：`session-2/code-my_logistic_regression.py`（标准实现）、`code-my_logistic_regression.md`（含讲解）；题库 `BagOfQuestions/BagOfQuestions-session-2-bh.md`（整段填空）、`-2-ak.md`（predict 填空 + 简答）

⭐ ⭐ 老师明确点名【必考】的具体代码行

> 出处：`session-2/code-my_logistic_regression.py`。以下三处是老师**明确说会考**的，必须默写：

```
# ① sigmoid（_sigmoid 方法）
return 1 / (1 + np.exp(-z))

# ② 线性模型（fit 与 predict 内）
linear_model = np.dot(X, self.weights) + self.bias

# ③ 参数更新（fit 循环内）
self.weights -= self.lr * dw
self.bias    -= self.lr * db
```

- ① **sigmoid 实现**：`1 / (1 + np.exp(-z))`——把分数压成概率。
- ② **线性模型**：`np.dot(X, self.weights) + self.bias`——即 z = XW + b。
- ③ **参数更新**：`self.weights -= self.lr * dw`、`self.bias -= self.lr * db`——梯度下降一步（`-=` 即 `W ← W − lr·dw`）。

> 配套要记得上文 `dw`、`db` 两行（`dw = (1/n)·np.dot(X.T,(y_predicted-y))`、`db = (1/n)·np.sum(y_predicted-y)`）才能让 ③ 成立——它们是 fit 的代码（要会写），只是**数学推导不考**。

英文原文（BoQ-2-bh，完整）

> ## Question: Logistic Regression from Scratch
>
> We are implementing binary logistic regression `class MyOwnLogisticRegression` from scratch with NumPy. The model receives a feature matrix `X` with shape `(n_samples, n_features)` and a binary target vector `y` with values in `{0, 1}`. First we compute the linear score $z = XW + \mathbf{1}b$, then we convert it to a probability with the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$.
>
> Fill in the `_____YOUR_CODE_HERE__N_____` blanks in the code skeleton below.

```
class MyOwnLogisticRegression:
    def __init__(self, learning_rate=0.001, n_iters=1000):
        self.lr = learning_rate
        self.n_iters = n_iters
        self.weights = None
        self.bias = None

    def _sigmoid(self, z):
        return ____YOUR_CODE_HERE__1_____

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(____YOUR_CODE_HERE__2_____)
        self.bias = ____YOUR_CODE_HERE__3_____
        for _ in range(self.n_iters):
            linear_model = np.dot(____YOUR_CODE_HERE__4_____, ____YOUR_CODE_HERE__5_____) + sel
            y_predicted = self._sigmoid(____YOUR_CODE_HERE__6_____)
            dw = (1 / n_samples) * np.dot(____YOUR_CODE_HERE__7_____, (y_predicted - y))
            db = (1 / n_samples) * np.sum(____YOUR_CODE_HERE__8_____)
            self.weights = self.weights - self.lr * ____YOUR_CODE_HERE__9_____
            self.bias = self.bias - self.lr * ____YOUR_CODE_HERE__10_____

    def predict(self, X):
        linear_model = np.dot(____YOUR_CODE_HERE__11_____, ____YOUR_CODE_HERE__12_____) + self.b
        y_predicted = self._sigmoid(____YOUR_CODE_HERE__13_____)
        y_predicted_cls = [1 if i > 0.5 else 0 for i in ____YOUR_CODE_HERE__14_____]
        return np.array(y_predicted_cls)
```

中文翻译

题目：从零实现逻辑回归

我们用 NumPy 从零实现二分类逻辑回归 `class MyOwnLogisticRegression`。模型输入特征矩阵 `X`，形状 `(n_samples, n_features)`，二分类标签 `y` 取值 `{0,1}`。先算线性分数 $z = XW + \mathbf{1}b$，再用 sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ 转成概率。

请填空 `____YOUR_CODE_HERE__N_____`（代码骨架同上）。

答案 Answer

English（考试作答）

```
1 :  1 / (1 + np.exp(-z))
2 :  n_features
3 :  0
4 :  X
5 :  self.weights
6 :  linear_model
7 :  X.T
8 :  (y_predicted - y)
9 :  dw
10:  db
11:  X
12:  self.weights
13:  linear_model
14:  y_predicted
```

中文解析（理解）

- 空 2 `n_features`：每个特征一个权重，所以 `weights` 长度 = 特征数；空 3 bias 初始化为 `0`。
- 空 4/5：`linear_model = np.dot(X, self.weights) + self.bias`，即线性分数 z = XW+b。
- 空 6：把 z 喂进 `_sigmoid` 得概率 `y_predicted`。
- 空 7/8：梯度 `dw = (1/n)·Xᵀ(ŷ−y)`、`db = (1/n)·Σ(ŷ−y)`（**这是 fit 的代码行，要会写**；其数学**推导不考**）。
- 空 9/10：参数更新 `W ← W − lr·dw`、`b ← b − lr·db`（**会考**）。
- 空 1：sigmoid 实现 `1/(1+np.exp(−z))`。

英文原文（BoQ-2-ak，完整）

> ## Question: Logistic Regression Predict Method
>
> We are implementing binary logistic regression `class MyOwnLogisticRegression` from scratch with NumPy. The model first computes a linear score, then calls its internal `_sigmoid` helper, and finally converts probabilities into class labels.

Fill in the `____YOUR_CODE_HERE__N_____` blanks in the `predict` method below.

```
def predict(self, X):
    linear_model = np.dot(____YOUR_CODE_HERE__1_____, ____YOUR_CODE_HERE__2_____) + self.bias
    y_predicted = self._sigmoid(____YOUR_CODE_HERE__3_____)
    y_predicted_cls = [1 if i > 0.5 else 0 for i in ____YOUR_CODE_HERE__4_____]
    return np.array(____YOUR_CODE_HERE__5_____)
```

Then answer the following short questions:

1. What is the difference between `linear_model`, `y_predicted`, and `y_predicted_cls`?
2. Why does this implementation call `self._sigmoid(...)` rather than a public `sigmoid(...)` method?
3. For probabilities `[0.2, 0.5, 0.8]`, what class labels does the shown threshold rule return? Explain the role of the strict `>` sign.

中文翻译

题目：逻辑回归 predict 方法

从零实现二分类逻辑回归：先算线性分数，再调内部 `_sigmoid`，最后把概率转成类别标签。请填空 predict 方法（骨架同上）。

然后回答简答：

1. `linear_model`、`y_predicted`、`y_predicted_cls` 三者有什么区别?
2. 为什么调用 `self._sigmoid(...)` 而不是公开的 `sigmoid(...)`?
3. 对概率 `[0.2, 0.5, 0.8]`，该阈值规则返回什么类别？解释严格 `>` 的作用。

答案 Answer　（⚠ predict 今日确认不考，仅供理解）

English（作答）

```
1: X    2: self.weights    3: linear_model    4: y_predicted    5: y_predicted_cls
```

Q1. `linear_model` is the raw linear score z = XW+b (range −∞..+∞); `y_predicted` is the probability after sigmoid (range 0..1); `y_predicted_cls` is the final 0/1 class label after

> thresholding.
>
> Q2. The leading underscore marks `_sigmoid` as a private/internal helper — an implementation detail, not part of the public API.
>
> Q3. `[0.2, 0.5, 0.8]` → `[0, 0, 1]`. The strict `>` means exactly 0.5 is **not** class 1 (0.5 → 0); only values strictly greater than 0.5 become 1.

中文解析

- 三者：分数 z → 概率 ŷ → 0/1 类别，逐级转换。
- `_sigmoid` 前缀下划线 = 私有/内部辅助函数，不对外暴露。
- 0.5 因为是严格 `>`，归到类 0；只有 >0.5 才是类 1。

考点 2.2　sigmoid 函数（会画图 + 公式）

文件：`session-2/lecture-2-logistic-regression-sigmoid-model.md` → "2. The sigmoid function" 、 "3. Why sigmoid?"

英文原文（完整节录）

> **1. The missing piece** — From the previous lecture, we have $z = xW + b$ ... But this is still $z \in (-\infty, +\infty)$. We need $\hat{y} \in [0, 1]$. So the question becomes: *How do we convert z into a valid probability?*
>
> **2. The sigmoid function** — We introduce a function: $\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$. This is called the sigmoid function.
>
> **3. Why sigmoid?** The sigmoid function has exactly the properties we need.
>
> - **Bounded output**: $\sigma(z) \in (0, 1)$. No matter what z is, the output is always a valid probability.
> - **Smooth and differentiable**: continuous and smooth everywhere. This makes it suitable for gradient-based optimization.
> - **Monotonic**: if $z_1 > z_2$, then $\sigma(z_1) > \sigma(z_2)$. So ordering is preserved.
> - **Key values**: $\sigma(0) = 0.5$.

中文翻译

> **缺失的一块**：上一讲得到 $z = xW + b$，但 $z \in (-\infty, +\infty)$，而我们要 $\hat{y} \in [0, 1]$。问题：怎么把 z 变成合法概率?
>
> **sigmoid 函数**：引入 $\sigma(z) = \frac{1}{1+e^{-z}}$。
>
> **为什么用 sigmoid**：恰好具备所需性质——**有界** $(0, 1)$（任何 z 都是合法概率）、**光滑可导**（适合梯度优化）、**单调**（保序）、**关键值** $\sigma(0) = 0.5$。

答案 Answer

English（作答：how to draw & key facts)

> Formula: σ(z) = 1 / (1 + e^(−z)). Graph: an S-shaped curve; horizontal asymptote y=0 as z→−∞ and y=1 as z→+∞; passes through (0, 0.5); monotonically increasing; steepest near z=0. Decision boundary: σ(z)=0.5 ⇔ z=0, so the boundary is z = xW+b = 0 (predict class 1 if z>0, else class 0).

中文解析

- **画图**：S 形曲线；左渐近 0、右渐近 1（两条水平渐近线）；过 (0, 0.5)，关于该点中心对称；中间最陡。
- **决策边界**：σ=0.5 ⇔ z=0 ⇔ 边界 `xW+b=0`。

考点 2.3　TensorFlow Playground 特征工程（BoQ-2-ag)

文件：`BagOfQuestions/BagOfQuestions-session-2-ag.md`；配套 `session-6/lecture-5-model-selection-bias-variance.md`（3. High Bias）

英文原文（完整）

> ## Question: TensorFlow Playground Feature Engineering for Curved and Ring-Shaped Boundaries

In a live TensorFlow Playground demo, we compare logistic regression behavior under different feature sets for binary classification with two raw inputs (x_1, x_2). We focus on curved geometry such as center-vs-ring patterns, where class $y = 0$ is concentrated near the origin and class $y = 1$ forms an outer ring.

1. Starting from raw features only, write the logistic regression score z and the decision boundary equation ($z = 0$). Explain geometrically why this setup underfits the ring-vs-center pattern.
2. For univariate regression, write degree-2 and degree-3 polynomial models, then the general degree-k form. Define a feature map $\phi(x)$ and explain why polynomial regression is still linear in parameters during optimization.
3. In TensorFlow Playground, enable engineered terms x_1^2 and x_2^2 so that $\phi(x) = [x_1, x_2, x_1^2, x_2^2]^T$. Write the transformed logistic score and boundary. State the qualitative sign pattern for weights/bias that would make $P(y = 1 \mid x)$ small near $(0, 0)$ and larger in outer regions.
4. Suppose the class geometry changes from concentric circles to (i) axis-aligned ellipses and (ii) rotated ellipses. For each case, identify which engineered terms are required and explain when the cross term $x_1 x_2$ becomes necessary.
5. Sketch three TensorFlow-Playground-style boundary outcomes on the original ring-vs-center dataset: (i) degree-1 underfitting, (ii) degree-2 good fit, and (iii) overly high-degree overfitting. Briefly interpret the bias-variance trade-off across the three sketches.

中文翻译

题目：TensorFlow Playground 中曲线/环形边界的特征工程

在 TF Playground 演示里，用两个原始输入 (x_1, x_2) 做二分类，比较不同特征集下逻辑回归的表现。重点是曲线几何，如“中心 vs 环”：$y = 0$ 集中在原点附近，$y = 1$ 形成外环。

1. 仅用原始特征，写出逻辑回归分数 z 和决策边界方程 $z = 0$。从几何上解释为什么它对“环 vs 中心”欠拟合。
2. 一元回归：写出 2 次、3 次多项式模型，再写一般 k 次形式。定义特征映射 $\phi(x)$，并解释为什么多项式回归在优化时对参数仍是线性的。
3. 在 Playground 启用工程项 x_1^2, x_2^2，使 $\phi(x) = [x_1, x_2, x_1^2, x_2^2]^T$。写出变换后的分数与边界。说明要让 $(0, 0)$ 附近 $P(y{=}1)$ 小、外圈大，权重/偏置的符号该如何。
4. 若几何从同心圆变成 (i) 轴对齐椭圆、(ii) 旋转椭圆。各需要哪些工程项？交叉项 $x_1 x_2$ 何时必需？

> 5. 在原“环 vs 中心”数据上画三种边界：(i) 1 次欠拟合、(ii) 2 次恰好、(iii) 过高次过拟合。简述三者的偏差-方差权衡。

答案 Answer

English（作答）

> 1. $z = w_1x_1 + w_2x_2 + b$; boundary $z=0$ is a straight line. A single line cannot enclose a center surrounded by a ring, so it underfits (high bias).
> 2. degree-2: $z = w_0 + w_1x + w_2x^2$; degree-3: $z = w_0 + w_1x + w_2x^2 + w_3x^3$; general degree-k: $z = \Sigma_{j=0}^{k} w_j x^j$. Feature map $\varphi(x) = [1, x, x^2, \ldots, x^k]$. It is linear in parameters because $z = w^T\varphi(x)$ is linear in w, so the same (linear) optimization applies.
> 3. $\varphi(x) = [x_1, x_2, x_1^2, x_2^2]$; $z = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + b$; boundary $z=0$ is a conic (circle/ellipse). To make P(y=1) small near origin and larger outside: $w_3, w_4 > 0$ (positive) and $b < 0$, so z increases with distance from the origin.
> 4. (i) axis-aligned ellipse → need x_1^2 and x_2^2 (with different weights). (ii) rotated ellipse → additionally need the cross term x_1x_2; it is necessary whenever the ellipse axes are not parallel to the coordinate axes.
> 5. degree-1: a line cutting through (underfit / high bias); degree-2: an ellipse matching the ring (good fit); high-degree: a wiggly boundary fitting noise (overfit / high variance). Bias–variance: as complexity ↑, bias ↓ but variance ↑; degree-2 is the sweet spot.

中文解析

- 原始特征 → 直线边界 → 包不住环 → 欠拟合（高偏差）。
- 多项式对**参数**线性：$z = w^T\phi(x)$，优化方法不变。
- 加 x_1^2, x_2^2 → 圆/椭圆边界；要中心概率低外圈高 → 平方项权重为正、bias 为负。
- **旋转椭圆必须加交叉项** x_1x_2（主轴不平行坐标轴时）。
- 三草图对应 欠拟合 / 恰好 / 过拟合，即偏差-方差权衡的 sweet spot 在 degree-2。

考点 2.4　加深网络 / 两螺旋（BoQ-2-ah）

文件：`BagOfQuestions/BagOfQuestions-session-2-ah.md`

英文原文（完整）

Question: Two Spirals in TensorFlow Playground and Representation Learning

Consider binary classification in TensorFlow Playground with the **Spiral** dataset, raw inputs (x_1, x_2), and sigmoid output probability $\hat{y} = \sigma(z)$.

1. **Stage 1 (Raw features, no hidden layer).** Use only x_1, x_2 and no hidden layers. Write the model score $z = w_1 x_1 + w_2 x_2 + b$, then explain why this boundary class cannot separate two intertwined spirals.
2. **Stage 2 (Manual feature engineering).** Add polynomial terms $x_1^2, x_2^2, x_1 x_2$, and optionally sinusoidal terms $\sin(x_1), \sin(x_2)$. Write an explicit transformed feature map $\phi(x)$ and a linear-in-parameters score $z = w^T \phi(x) + b$. Explain the expected effect: partial improvement and richer curvature, but still limited global spiral separation.
3. **Stage 3 (Add depth).** Keep the same input features and add hidden layers with ReLu activations, for example one hidden layer (6--8 neurons), then two or three layers (about 8--12 neurons each). Explain how compositions $x \to h^{(1)} \to h^{(2)} \to \cdots \to \hat{y}$ can be interpreted as progressively transforming coordinates so classes become easier to separate.
4. Draw and label a four-panel progression of decision boundaries for: (i) raw logistic model, (ii) engineered features only, (iii) shallow network, and (iv) deeper network. For each panel, add one sentence describing what geometric structure is captured and what remains incorrect.
5. Write a short concluding paragraph (3--5 sentences): Why is the two-spirals experiment a strong argument for deep learning, and why is "deep works because of learned representations" a stronger explanation than "deep works because it has many parameters"?

中文翻译

题目：TensorFlow Playground 两螺旋与表示学习

在 Playground 用 **Spiral（螺旋）** 数据集做二分类，原始输入 (x_1, x_2)，sigmoid 输出概率 $\hat{y} = \sigma(z)$。

1. **阶段1（原始特征、无隐藏层）**：只用 x_1, x_2。写出 $z = w_1x_1 + w_2x_2 + b$，并解释为什么这种（线性）边界分不开两条交缠的螺旋。
2. **阶段2（手工特征工程）**：加入 x_1^2, x_2^2, x_1x_2，可选 $\sin(x_1), \sin(x_2)$。写出显式特征映射 $\phi(x)$ 与对参数线性的分数 $z = w^T\phi(x) + b$。说明预期效果：部分改善、曲率更丰富，但全局仍难分开螺旋。
3. **阶段3（加深度）**：保持输入特征不变，加 ReLU 隐藏层（如 1 层 6–8 神经元，再 2–3 层各约 8–12 个）。解释复合 $x \to h^{(1)} \to h^{(2)} \to \cdots \to \hat{y}$ 如何被理解为“逐层变换坐标”使类别更易分开。
4. 画并标注四格决策边界演进：(i) 原始逻辑回归、(ii) 仅工程特征、(iii) 浅层网络、(iv) 更深网络。每格一句话说明捕捉到什么几何结构、还有什么不对。
5. 写 3–5 句结论：为什么两螺旋实验是支持深度学习的有力论据？为什么“深度有效是因为学到了表示”比“深度有效是因为参数多”更强？

答案 Answer

English（作答）

1. $z = w_1x_1 + w_2x_2 + b$. The boundary $z=0$ is a straight line; two intertwined spirals are highly non-linear and cannot be separated by any single line, so raw logistic regression fails.
2. $\varphi(x) = [x_1, x_2, x_1^2, x_2^2, x_1x_2, \sin(x_1), \sin(x_2)]$; $z = w^T\varphi(x) + b$. This adds curvature and gives partial improvement, but hand-picked features still cannot globally separate the spirals.
3. With ReLU hidden layers, $x \to h^{(1)} \to h^{(2)} \to \ldots \to \hat{y}$. Each layer applies a non-linear coordinate transformation; stacking them progressively warps the input space until the two spirals become (almost) linearly separable in the final hidden representation.
4. (i) raw logistic: a straight line — misses the spiral structure entirely; (ii) engineered features: a curved boundary — captures some curvature but not the full intertwining; (iii) shallow network: bends around part of the spirals but still leaks; (iv) deeper network: follows both spiral arms closely — correct separation.
5. The two-spirals task needs a highly non-linear boundary, which raw/linear models cannot produce, so success with depth argues strongly for deep learning. “Learned representations” is the stronger explanation because depth works by learning hierarchical features that reshape the input space into a separable one — not merely by having many parameters (a wide shallow model with many parameters still struggles). Representation learning, not raw parameter count, is the source of the power.

中文解析

- 阶段1：线性边界分不开高度非线性的螺旋。
- 阶段2：手工特征只能部分改善，全局仍不行。
- 阶段3：ReLU 隐藏层 = 逐层非线性变换坐标，最终在隐藏表示里把螺旋拉成可分。
- 四格图：直线 → 曲线 → 浅层 → 深层，越来越贴合两条螺旋臂。
- 结论关键句：**深度强在“学到分层表示、重塑空间”，而非“参数多”**（同样多参数的宽浅模型仍难分开）。

Session 2 不考清单（今日确认）

- ❌ BCE 二元交叉熵、CE vs BCE 区别
- ❌ dw / db 的**数学推导**（代码行仍要会）
- ❌ predict 函数（主考 fit；2-ak 仅供理解）
- ❌ sigmoid 的独立代码（但要会在 fit 里调用 + 会画 sigmoid 图）
- ↪ softmax：本章常同题出现，详细整理放在 **Session 3**。

ML01 期末复习 · 详细整理 —— Session 3 神经网络 Neural Networks

> 格式：文件索引/位置 → **英文原文（完整）+ 中文翻译 → 答案（English 作答 + 中文解析）**。
>
> 今日确认：softmax / 激活函数(**重点 sigmoid**) / 前向传播 / 参数量统计 / FFN·MLP **考**；CE / BCE / MSE 损失公式本身 **不考**；one-hot 仅「为何 10 logit + 写 one-hot 向量」要会、输出层「每任务的输出神经元数 + 激活」要会（见 3.6）。

考点 3.1　前向传播 Forward Propagation

文件：`session-3/lecture-3-neural-networks-forward-propagation.md`；代码 `lecture-8-code-nn-forward-propagation-numpy_matrix_operations.ipynb`

英文原文（完整）

> **1. Single-Layer Recap** — For a single layer: $a = f(xW + b)$
>
> - $x \in \mathbb{R}^{1\times d}$ input row vector; $W \in \mathbb{R}^{d\times n}$ weight matrix; $b \in \mathbb{R}^{1\times n}$ bias row vector; f activation; $a \in \mathbb{R}^{1\times n}$ output activation. This is the fundamental building block for deeper networks.
>
> **2. Multi-Layer Recursion** — For layer l in a network with L layers: $z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$; $a^{(l)} = f^{(l)}(z^{(l)})$. With $a^{(0)} = x$, repeat for $l = 1, 2, \ldots, L$.
>
> **3. Final Output** — $\hat{y} = a^{(L)}$. This could be a scalar (regression), a probability (binary classification), or a probability vector (multiclass classification).
>
> **4. Intuition** — Each layer transforms the representation of the input: early layers → low-level features; middle → intermediate patterns; later → high-level abstract representation. Forward propagation is just applying linear + nonlinear transformations repeatedly.

中文翻译

> **单层回顾**：单层为 $a = f(xW + b)$（x 输入行向量、W 权重、b 偏置、f 激活、a 输出）。这是更深网络的基本积木。
>
> **多层递推**：第 l 层 $z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$，$a^{(l)} = f^{(l)}(z^{(l)})$；$a^{(0)} = x$，对 $l = 1..L$ 重复。
>
> **最终输出**：$\hat{y} = a^{(L)}$——可以是标量(回归)、概率(二分类)或概率向量(多分类)。
>
> **直觉**：每层都在**变换输入的表示**（浅层→低级特征，中层→中间模式，深层→高级抽象）；前向传播就是反复做“线性+非线性”变换。

答案 Answer

English（作答：必会写的公式）

> Single layer: a = f(xW + b). Layer l: $z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$, $a^{(l)} = f^{(l)}(z^{(l)})$, with $a^{(0)} = x$. Output: $\hat{y} = a^{(L)}$.

中文解析

- 前向传播 = 从输入 $x = a^{(0)}$ 起，逐层算 “线性 z → 激活 a”，到最后一层得 $\hat{y}$。
- 关键点：每层先线性 `aW+b` 再过激活 `f`；输出层形式由任务决定（回归/二分类/多分类）。

考点 3.2　softmax（公式 + 计算 + 平移不变性 + 数值稳定）

文件：`session-3/lecture-5-neural-networks-output-layers-and-softmax.md`、`lecture-6-softmax-deep-dive.md`；题库 `BoQ-3-ae`（计算）、`BoQ-3-af`（平移不变 + 数值稳定）

英文原文（BoQ-3-ae，完整）

> ## Question: Softmax Calculation and Probability Interpretation
>
> Consider a three-class neural network classifier that outputs a logit vector $z = [z_1, z_2, z_3] = [2, 1, 0]$ for a given input example.

1. Write the general mathematical formula for the softmax function used to calculate the predicted probability p_k for a specific class k.
2. Using the approximations $e^2 \approx 7.39$, $e^1 \approx 2.72$, and $e^0 = 1.00$, compute the numeric value of the normalization denominator (the sum of the exponentials).
3. Compute the three final softmax probabilities $[p_1, p_2, p_3]$ rounded to two decimal places. Verify that your calculated probabilities sum exactly to 1.00.
4. Determine which class index is selected by an `argmax` operation over the probabilities. Sketch two simple side-by-side bar charts: one displaying the raw logit scores z, and the other displaying the resulting probabilities p.
5. Suppose a different input example yields a shifted logit vector $z_{\text{new}} = [102, 101, 100]$. State the resulting probability vector p_{new} without performing explicit exponentiation. Use this scenario to explain why the softmax function depends entirely on the relative differences between logit values rather than their absolute magnitudes.

中文翻译

题目：softmax 计算与概率解释

三分类网络对某输入输出 logits $z = [2, 1, 0]$。

1. 写出 softmax 计算第 k 类概率 p_k 的一般公式。
2. 用 $e^2 \approx 7.39, e^1 \approx 2.72, e^0 = 1.00$，算归一化分母（指数之和）。
3. 算三个概率 $[p_1, p_2, p_3]$（保留两位小数），验证和为 1.00。
4. 用 argmax 选出哪一类。并排画两个柱状图：一个是原始 logits z，一个是概率 p。
5. 若另一输入得 $z_{new} = [102, 101, 100]$，不做指数运算直接说出 p_{new}，并解释为什么 softmax 完全取决于 logits 之间的相对差、而非绝对大小。

答案 Answer（BoQ-3-ae）

English（作答）

1. p_k = exp(z_k) / Σ_j exp(z_j).
2. Denominator = $e^2 + e^1 + e^0 \approx 7.39 + 2.72 + 1.00 = 11.11$.
3. $p_1 = 7.39/11.11 \approx 0.67$; $p_2 = 2.72/11.11 \approx 0.24$; $p_3 = 1.00/11.11 \approx 0.09$. Sum = 0.67+0.24+0.09 = 1.00. ✓

4. argmax → class index 1 (the first class, z_1=2). Bar charts: logits [2,1,0] (linearly spaced); probabilities [0.67,0.24,0.09] (first bar dominant).
5. p_new = [0.67, 0.24, 0.09], identical to before, because [102,101,100] has the same pairwise differences as [2,1,0]. Softmax is shift-invariant: adding the same constant to all logits cancels in numerator and denominator, so only the relative differences matter.

中文解析

- 分母 = 三个指数之和 ≈ 11.11；概率 = 各指数 / 分母。
- argmax 选最大 logit 对应的类（这里第 1 类）。
- **平移不变性**：所有 logits 同加常数，softmax 不变 → 只看相对差，所以 [102,101,100] 与 [2,1,0] 概率相同。

英文原文（BoQ-3-af，完整）

Question: Softmax Shift Invariance and Numerical Stability

Softmax has the important property: $\text{softmax}(z) = \text{softmax}(z + c)$ where the same constant c is added to every logit.

1. Prove this property using the softmax formula.
2. Explain why $[2, 1, 0]$ and $[102, 101, 100]$ give the same softmax probabilities. Explain the numerical danger of applying exponential directly to logits like $[1000, 999, 998]$.
3. Show how subtracting the maximum transforms $[1000, 999, 998]$ into a safer vector.
4. Write the stable softmax formula using $z_i - \max(z)$.

中文翻译

题目：softmax 平移不变性与数值稳定

softmax 有重要性质 $\text{softmax}(z) = \text{softmax}(z + c)$（每个 logit 加同一常数 c）。

1. 用 softmax 公式证明该性质。

2. 解释为什么 $[2,1,0]$ 与 $[102,101,100]$ 概率相同。解释直接对 $[1000,999,998]$ 求指数的数值危险。
3. 演示减最大值如何把 $[1000,999,998]$ 变成更安全的向量。
4. 写出用 $z_i - \max(z)$ 的数值稳定 softmax 公式。

答案 Answer（BoQ-3-af）

English（作答）

1. softmax(z+c)_i = exp(z_i + c) / Σ_j exp(z_j + c) = (e^c · e^{z_i}) / (e^c · Σ_j e^{z_j}) = e^{z_i} / Σ_j e^{z_j} = softmax(z)_i. The e^c cancels. ■
2. [2,1,0] and [102,101,100] differ by a constant shift of 100; since softmax depends only on differences, the probabilities are identical. Danger: e^{1000} overflows floating point (→ inf), giving inf/inf = NaN.
3. Subtract max = 1000: [1000,999,998] − 1000 = [0, −1, −2], whose exponentials e^0, e^{-1}, e^{-2} are all small and safe.
4. Stable softmax: p_i = exp(z_i − max(z)) / Σ_j exp(z_j − max(z)).

中文解析

- 证明：同加 c → 分子分母都乘 e^c → 约掉 → 不变。
- e^{1000} 会**溢出**变 inf；减最大值后指数都 ≤ 1，安全（结果数学上不变，因平移不变性）。

考点 3.3　激活函数 Activation Functions

文件：`session-3/lecture-4-neural-networks-activation-functions.md`；题库 `BoQ-3-ac`

考试重点：激活函数里**重点只掌握 sigmoid**（公式 + 画图）；ReLU/tanh 内容不难、了解即可。**第 5 问（只堆线性层、无激活 → 等价单个线性层）较可能考，需会答。**

英文原文（BoQ-3-ac，完整）

Question: Activation Functions

A neural network repeatedly applies linear transformations and activation functions.

1. Explain why activation functions introduce nonlinearity.
2. Draw the ReLU activation function. Write the formula for ReLU.
3. Draw the sigmoid activation function. Write the formula for sigmoid.
4. Draw the tanh activation function and mark its output range.
5. If a network stacks many linear layers but uses no activation functions, what simpler model is it equivalent to? Explain why this would be bad for learning complex image or text patterns.

中文翻译

题目：激活函数

神经网络反复做“线性变换 + 激活函数”。

1. 解释激活函数为何引入非线性。
2. 画 ReLU，并写公式。
3. 画 sigmoid，并写公式。
4. 画 tanh，并标注输出范围。
5. 若网络堆叠很多线性层但**不用激活函数**，它等价于什么更简单的模型？为什么这对学习复杂图像/文本模式不利？

答案 Answer（BoQ-3-ac)

English（作答）

1. Activation functions are non-linear; inserting them between linear layers lets the network represent non-linear functions. Without them, the model can only represent linear mappings.
2. ReLU: f(z) = max(0, z). Graph: flat 0 for z<0, then the line y=z for z≥0 (a "hinge" at the origin).
3. Sigmoid: σ(z) = 1 / (1 + e^{−z}). Graph: S-curve in (0,1), passing (0, 0.5).
4. tanh: f(z) = (e^z − e^{−z}) / (e^z + e^{−z}). Graph: S-curve through the origin; output range (−1, 1).
5. Stacking linear layers with no activation is equivalent to a single linear layer (one linear transformation), because a composition of linear maps is linear. This is bad: a

> single linear model cannot capture the complex non-linear patterns in images or text.

中文解析

- 没有激活 → 线性套线性还是线性 → 等价**一个**线性层；表达力受限。
- ReLU=max(0,z)（折线）；sigmoid 在 (0,1)；tanh 在 (−1,1) 且过原点。

考点 3.4　参数量统计 Parameter Counting（⭐ ⭐ 重点，统计含 bias）

文件：题库 `BoQ-3-an`（tiny LM）、`BoQ-3-ag`（MoE）、`BoQ-3-ap`（权重共享）、`BoQ-3-bm`（规模对比）

> 通用法则：一个线性/dense 层 `Linear(in, out)` 参数 = **权重 in×out + 偏置 out**。题目都明确“one bias per output unit”，所以 **bias 要算**。

英文原文（BoQ-3-an，完整）

> ### Question: ChatGPT-Style Tiny Language Model Parameter Counting
>
> A company uses a tiny ChatGPT-style demo... The vocabulary has $V = 1000$ possible tokens. Each token is represented by a vector of length $d = 10$.
>
> The toy model has three trainable parts: an embedding table $E \in \mathbb{R}^{1000\times 10}$, one hidden dense layer with weight matrix $W_h \in \mathbb{R}^{10\times 20}$ and bias vector $b_h \in \mathbb{R}^{1\times 20}$, and an output vocabulary layer with weight matrix $W_{out} \in \mathbb{R}^{20\times 1000}$ and bias vector $b_{out} \in \mathbb{R}^{1\times 1000}$.
>
> 1. Count the trainable parameters in the embedding table. Explain in one sentence what this table stores.
> 2. Count the weights and biases in the hidden dense layer.
> 3. Count the weights and biases in the output vocabulary layer.
> 4. Add the three counts to get the total number of trainable parameters in this toy model.
> 5. Explain why the output layer is large even though the hidden layer is small.

中文翻译

> ## 题目：ChatGPT 风格微型语言模型参数量
>
> 词表 $V = 1000$，每个 token 向量长 $d = 10$。模型三部分：embedding 表 $E \in \mathbb{R}^{1000\times 10}$；隐藏层 $W_h \in \mathbb{R}^{10\times 20}, b_h \in \mathbb{R}^{1\times 20}$；输出词表层 $W_{out} \in \mathbb{R}^{20\times 1000}, b_{out} \in \mathbb{R}^{1\times 1000}$。
>
> 1. 数 embedding 表参数，并一句话说明它存什么。
> 2. 数隐藏层的权重与偏置。
> 3. 数输出层的权重与偏置。
> 4. 三者相加得总参数。
> 5. 解释为什么隐藏层小但输出层大。

答案 Answer（BoQ-3-an）

English（作答）

> 1. Embedding: 1000 × 10 = 10,000 parameters. It stores one d-dim vector for each of the V tokens.
> 2. Hidden layer: weights 10 × 20 = 200, bias 20 → 220.
> 3. Output layer: weights 20 × 1000 = 20,000, bias 1000 → 21,000.
> 4. Total = 10,000 + 220 + 21,000 = 31,220.
> 5. The output layer maps the hidden width (20) to the full vocabulary V=1000 (one logit per token), so its weight matrix (20×1000) dominates, while the hidden layer only maps 10→20.

中文解析：embedding=V·d；隐藏=10·20+20；输出=20·1000+1000；输出层大是因为要对**整个词表**每个 token 出一个 logit。

英文原文（BoQ-3-ag，完整）

> ## Question: DeepSeek Mini Mixture-of-Experts
>
> A simplified DeepSeek-inspired model uses a tiny mixture-of-experts layer... There are 4 experts. Each expert is a small two-layer network: first a dense layer from 10 inputs to 100

hidden units, then a dense layer from 100 hidden units back to 10 outputs. Each dense layer has one bias per output unit. A router chooses experts using a dense layer from 10 inputs to 4 expert scores, also with one bias per output score.

1. Count the parameters in the first dense layer of one expert.
2. Count the parameters in the second dense layer of one expert, then compute the total parameters in one expert.
3. Count the parameters in all 4 experts together.
4. Count the parameters in the router and then compute the total stored trainable parameters in this mixture-of-experts layer.
5. If only 2 of the 4 experts are used for one token, explain why the model still stores parameters for all 4 experts.

中文翻译

题目：DeepSeek 迷你 MoE

4 个 expert，每个是两层网络：dense 10→100，再 dense 100→10；每层每输出一个 bias。router 用 dense 10→4 选 expert，也每输出一个 bias。

1. 一个 expert 第一层 dense 的参数。
2. 第二层 dense 的参数，再算一个 expert 总参数。
3. 4 个 expert 合计。
4. router 参数，再算整个 MoE 层存储的总可训练参数。
5. 若一个 token 只用 4 个里的 2 个 expert，为什么仍要存全部 4 个的参数?

答案 Answer（BoQ-3-ag）

English（作答）

1. First dense (10→100): 10 × 100 + 100 = 1,100.
2. Second dense (100→10): 100 × 10 + 10 = 1,010. One expert total = 1,100 + 1,010 = 2,110.
3. All 4 experts: 4 × 2,110 = 8,440.
4. Router (10→4): 10 × 4 + 4 = 44. Total MoE layer = 8,440 + 44 = 8,484.
5. The router decides per token which experts to use, and different tokens activate different experts, so all 4 experts must be stored and trainable even though only 2

are used for any single token.

中文解析：每层 in×out+out；4 expert 共 8440 + router 44 = 8484。**虽每 token 只激活部分，但要为所有 token 准备 → 全部 expert 都得存。**

英文原文（BoQ-3-ap，完整）

Question: ChatGPT Vocabulary Table and Shared Weights

... vocabulary $V = 1000$, token vector length $d = 10$. The model has an embedding table $E \in \mathbb{R}^{1000\times 10}$.

At the end, the model needs one logit for each possible next token. One design uses a separate output matrix $W_{out} \in \mathbb{R}^{10\times 1000}$ and an output bias $b_{out} \in \mathbb{R}^{1\times 1000}$. Another design reuses the embedding table as the output matrix, so there is no separate W_{out}, but the output bias is still trainable.

1. Count the parameters in the embedding table E.
2. Count the parameters in the separate output matrix W_{out} and output bias b_{out}.
3. If the model reuses the embedding table and keeps only the output bias as extra output parameters, how many output-side parameters remain?
4. How many parameters are saved by reusing the embedding table instead of storing a separate W_{out}?
5. Explain why the model needs 1000 output logits when the vocabulary has 1000 tokens.

中文翻译

题目：ChatGPT 词表与权重共享

词表 $V = 1000$，token 向量长 $d = 10$，embedding 表 $E \in \mathbb{R}^{1000\times 10}$。末端需对每个候选 token 出一个 logit。设计一：单独输出矩阵 $W_{out} \in \mathbb{R}^{10\times 1000}$+偏置 b_{out}。设计二：复用 embedding 当输出矩阵（无单独 W_{out}），但输出偏置仍可训练。

1. embedding 参数。
2. 单独 W_{out} 和 b_{out} 参数。

3. 若复用 embedding、只留输出偏置，输出侧还剩多少参数?
4. 复用 embedding 相比单独 W_{out} 省了多少参数?
5. 为什么词表 1000 就需要 1000 个输出 logit?

答案 Answer（BoQ-3-ap）

English（作答）

1. Embedding E: 1000 × 10 = 10,000.
2. Separate W_out: 10 × 1000 = 10,000; b_out = 1000 → 11,000 total.
3. Reusing the embedding, only the output bias remains: 1,000 output-side parameters.
4. Saved = the W_out matrix = 10 × 1000 = 10,000 parameters.
5. The model predicts a probability distribution over the next token; with 1000 possible tokens it needs 1000 logits (one score per token) before softmax.

中文解析：权重共享（weight tying）= 复用 embedding 当输出矩阵 → **省下 W_out 的 10,000**，只剩 bias 1000。要 1000 logit 是因为要对 1000 个候选 token 各打一个分。

英文原文（BoQ-3-bm，完整）

Question: DeepSeek-Style Model Size Comparison

... Both models use a vocabulary of 1000 tokens and output one score for each possible next token.

Model A uses hidden width 10. It has one dense layer from 10 inputs to 20 hidden units, followed by an output dense layer from 20 hidden units to 1000 vocabulary scores. Model B doubles these internal sizes: one dense layer from 20 inputs to 40 hidden units, followed by an output dense layer from 40 hidden units to 1000 vocabulary scores. Every dense layer has one bias per output unit.

1. For Model A, count the parameters in the first dense layer and in the output layer.
2. For Model B, count the parameters in the first dense layer and in the output layer.
3. Compute the total counted parameters for Model A and for Model B.
4. Which part is larger in each model: the small internal dense layer or the output vocabulary layer? Explain using the numbers.

中文翻译

> ### 题目：DeepSeek 风格模型规模对比
>
> 两模型词表都 1000。模型 A：dense 10→20，输出 dense 20→1000。模型 B（内部翻倍）：dense 20→40，输出 dense 40→1000。每层每输出一个 bias。
>
> 1. 模型 A：第一层与输出层参数。
> 2. 模型 B：第一层与输出层参数。
> 3. 各自总参数。
> 4. 每个模型里：小的内部 dense 层 vs 输出词表层，哪个更大？用数字解释。

答案 Answer（BoQ-3-bm）

English（作答）

> 1. Model A — first dense (10→20): 10×20 + 20 = 220; output (20→1000): 20×1000 + 1000 = 21,000.
> 2. Model B — first dense (20→40): 20×40 + 40 = 840; output (40→1000): 40×1000 + 1000 = 41,000.
> 3. Total: Model A = 220 + 21,000 = 21,220; Model B = 840 + 41,000 = 41,840.
> 4. In both models the output vocabulary layer is far larger (21,000 ≫ 220; 41,000 ≫ 840), because it projects to the large vocabulary V=1000, which dominates the parameter count.

中文解析：输出层(乘 V=1000)远大于内部小层；模型规模主要由词表输出层决定。

考点 3.5　FFN / MLP（含义 + 结构 + 隐藏层数）

文件：`session-223-ffn-mini-series/lecture-1-ffn-structure-and-role.md`

英文原文（完整节录）

2. What is the FFN? — The Structure — A standard FFN consists of: (1) Linear expansion: project to higher dimension (typically 4×); (2) Non-linear activation: element-wise non-linearity; (3) Linear contraction: project back to model dimension. Mathematically: $\mathrm{FFN}(x) = \mathrm{activation}(xW_1 + b_1)W_2 + b_2$, where $W_1 \in \mathbb{R}^{d_{model} \times 4d_{model}}$ (expansion), $W_2 \in \mathbb{R}^{4d_{model} \times d_{model}}$ (contraction).

3. Why Expand Then Contract? — Input 768-dim → [3072-dim intermediate] → 768-dim. Why 4×: increased capacity, overcomplete representation, empirically optimal.

4. Position Independence — Unlike attention, FFN is applied identically to every position; no information flows between positions. Separation of concerns: **Attention** = "Where should I look?" (routing); **FFN** = "What should I do with what I found?" (processing).

中文翻译

FFN 结构：(1) 线性扩展（升到约 4×维）；(2) 逐元素非线性激活；(3) 线性收缩（回到 d_model）。公式 $\mathrm{FFN}(x) = \mathrm{activation}(xW_1 + b_1)W_2 + b_2$；$W_1 : d_{model} \times 4d_{model}$、$W_2 : 4d_{model} \times d_{model}$。

为何先扩后缩：768→3072→768；4× 是为提升容量、过完备表示、经验最优。

逐位置独立：FFN 对每个 token 用同一套权重、位置间不交换信息。分工：**attention 负责“看哪里”(路由)，FFN 负责“处理信息”**。

答案 Answer

English（作答）

FFN (Feed-Forward Network) is the same as an MLP (Multi-Layer Perceptron). Structure: FFN(x) = activation($xW_1 + b_1$)$W_2 + b_2$, expanding d_model → 4·d_model → d_model. It has exactly one hidden layer (the 4×d_model intermediate). Applied independently per token; attention routes, FFN processes.

中文解析

- **FFN = MLP**；结构 = 线性升维(4×) → 激活 → 线性降维。

- **隐藏层只有 1 个**（中间的 4×d_model 层）。
- 参数量（配合 3.4）：FFN ≈ d·4d + 4d·d = 8d²；attention(Q/K/V/O) ≈ 4d² → 每个 block 里 **FFN 约是 attention 的 2 倍。**

考点 3.6　输出层与 one-hot（题面优先收录，含被划走的损失片段）

> 这两题主体（损失 MSE/BCE/CE）不考，但题面里夹着**要考的片段**，按“题面优先”仍收录。

BoQ-3-ah　Output Layers Depend on the Task（⭐ 重点：每种任务的输出神经元个数 + 激活）

文件：题库 `BoQ-3-ah`

英文原文（完整）

> ## Question: Output Layers Depend on the Task
>
> A neural network's final layer must match the prediction task. Under the row-vector convention, assume the last hidden representation for one example is $a^{(L-1)}$, and the final affine transformation is $z^{(L)} = a^{(L-1)}W^{(L)} + b^{(L)}$.
>
> For each case below, write the appropriate output dimension, the usual final activation, and the typical loss formula.
>
> 1. **One-output regression.** The target is one continuous number $y \in \mathbb{R}$. What output dimension and final activation should be used? Write the mean squared error loss for one example.
> 2. **Binary classification.** The target is $y \in \{0, 1\}$. If we use one logit, what output dimension and activation should be used? Write the sigmoid formula $\sigma(z)$ and the binary cross-entropy loss for one example.
> 3. **Multiclass classification with 10 classes.** The target class is $c \in \{0, 1, \ldots, 9\}$. What output dimension and activation should be used? Write the softmax formula for class k and the cross-entropy loss for one example.

中文翻译

> ## 题目：输出层取决于任务
>
> 网络最后一层要匹配预测任务。行向量约定下，最后隐藏表示为 $a^{(L-1)}$，末端仿射变换 $z^{(L)} = a^{(L-1)}W^{(L)} + b^{(L)}$。对每种情形，写出**输出维度、常用末端激活、典型损失公式。**
>
> 1. **单输出回归**：目标 $y \in \mathbb{R}$。输出维度与激活？写出单样本 MSE。
> 2. **二分类**：$y \in \{0, 1\}$，用一个 logit。输出维度与激活？写出 $\sigma(z)$ 与单样本 BCE。
> 3. **10 类多分类**：$c \in \{0, \ldots, 9\}$。输出维度与激活？写出第 k 类 softmax 与单样本 CE。

答案 Answer（⭐ 重点在“输出维度 = 输出神经元数”与激活；loss 公式不考，仅列全）

English（作答）

> 1. Regression: output dim = **1**; activation = **none (identity/linear)**. MSE = (ŷ − y)².
> 2. Binary: output dim = **1**; activation = **sigmoid**, σ(z) = 1/(1 + e^{−z}). BCE = −[y·log ŷ + (1−y)·log(1−ŷ)].
> 3. Multiclass (10): output dim = **10**; activation = **softmax**, softmax(z)_k = e^{z_k} / Σ_j e^{z_j}. CE = −Σ_k y_k·log ŷ_k.

中文解析

- ⭐ **核心考点 = 输出层神经元个数（= 输出维度）**：回归 **1**、二分类 **1**、K 分类 **K**（此处 10）。
- 激活：回归**无**（线性）/ 二分类 **sigmoid** / 多分类 **softmax** —— 这些要会。
- ❌ 三个 loss（MSE / BCE / CE）公式本身**不考**，上面仅为完整性列出。

BoQ-3-aj　One-Hot Labels（只收录第 1 问）

文件：题库 `BoQ-3-aj`

英文原文（第 1 问）

> A digit classifier has $K = 10$ possible classes (digits 0–9). The model outputs logits $z \in \mathbb{R}^{1\times 10}$ and softmax probabilities $\hat{y} \in \mathbb{R}^{1\times 10}$.

> 1. Explain why the output layer needs 10 logits. If the true digit is 3, describe the one-hot target vector $y \in \mathbb{R}^{1\times 10}$.

中文翻译

> 数字分类器有 $K = 10$ 类（0–9），输出 logits $z \in \mathbb{R}^{1\times 10}$、softmax 概率 $\hat{y} \in \mathbb{R}^{1\times 10}$。
>
> 1. 解释为何输出层需要 10 个 logit。若真实数字是 3，描述 one-hot 目标向量 $y \in \mathbb{R}^{1\times 10}$。

答案 Answer

English（作答）

> 10 classes (digits 0–9) → one logit per class → **10 logits**. For true digit 3, the one-hot target is y = [0,0,0,1,0,0,0,0,0,0] — a 1 at index 3 (0-indexed) and 0 elsewhere.

中文解析

- 10 类各对应一个 logit → 输出 **10** 个。
- 数字 3 的 one-hot：第 3 位（从 0 数）为 1，其余为 0 → [0,0,0,1,0,0,0,0,0,0]。
- 注：该题第 2–4 问涉及 CE 计算，**不考**，故不收录。

Session 3 不考清单（今日确认）

- ❌ CE / BCE / MSE 损失函数公式本身：不考（含多分类 CE）。
- ⚠️ one-hot（BoQ-3-aj）：仅**第 1 问**（为何 10 logit + 写 one-hot 向量）要会，已收录于 3.6；第 2–4 问（CE 计算）不考。
- ⚠️ 输出层按任务（BoQ-3-ah）：**输出神经元数 + 激活**要会，已收录于 3.6（重点）；三种 loss 公式不考。
- 注：softmax **考**（公式/计算/平移不变/数值稳定），只是它常配的损失 CE 不单独考。

ML01 期末复习 · 详细整理 —— Session 5 优化器 Optimization（重点章节•主考数学公式）

> 格式：文件索引/位置 → **英文原文（完整）+ 中文翻译 → 答案（English 作答 + 中文解析）**。
>
> 范围确认：本章是**重点**，主考**数学更新公式**（GD/SGD、Momentum、Adam）+ **会画更新轨迹图**。 核心口径：Adam 是本课程默认/唯一优化器（"Adam is used everywhere"），与大模型训练相关。
>
> 题库映射：5-aa（梯度→更新） | 5-ab（batch size） | 5-ac（full-batch vs mini-batch） | 5-ad（Adam bias correction） | 5-ae（Momentum） | 5-af（Adam 全套公式） | 5-ag（四算法对比+画图）。

考点 5.1　从梯度到参数更新 Gradient → Update（GD 基本公式 + 符号 + 计算）

文件：`session-5/lecture-1-optimization-from-gradient-to-update.md`；题库 `BoQ-5-aa`

英文原文（BoQ-5-aa，完整）

> ### Question: From Gradient to Parameter Update
>
> During neural-network training, suppose the loss over the current mini-batch is $\mathcal{L}$ and the gradient for a weight matrix is $g = \frac{\partial \mathcal{L}}{\partial W}$. The learning rate is η, and gradient descent uses the update notation $W \leftarrow W - \eta g$.
>
> 1. Explain why computing a gradient is not the same thing as learning.
> 2. In one sentence, explain why the update moves in the negative gradient direction. Draw a one-dimensional loss curve and label the current parameter value, the gradient direction, the negative-gradient direction, and the updated parameter value.
> 3. Explain the role of the learning rate η. If one scalar weight has value $W = 2.0$, gradient $g = 0.5$, and learning rate $\eta = 0.1$, compute the updated value of W.

4. If one scalar weight has value $W = 2.0$, gradient $g = -0.5$, and learning rate $\eta = 0.1$, compute the updated value of W. Explain why the sign of the gradient changes the update direction.

中文翻译

题目：从梯度到参数更新

训练中，设当前 mini-batch 上的损失为 $\mathcal{L}$，权重矩阵的梯度 $g = \frac{\partial \mathcal{L}}{\partial W}$，学习率 η，梯度下降更新记为 $W \leftarrow W - \eta g$。

1. 解释为何「算出梯度」不等于「学习」。
2. 一句话解释为何更新沿**负梯度**方向。画一维损失曲线，标注当前参数、梯度方向、负梯度方向、更新后参数。
3. 解释学习率 η 的作用。若标量权重 $W = 2.0$、$g = 0.5$、$\eta = 0.1$，算更新后的 W。
4. 若 $W = 2.0$、$g = -0.5$、$\eta = 0.1$，算更新后的 W，并解释为何梯度符号改变更新方向。

答案 Answer（BoQ-5-aa）

English（作答）

1. The gradient only tells the **direction** of steepest increase of the loss at the current point; learning is the **act of updating** the parameters using that gradient (and repeating). A gradient alone changes nothing until an update step is applied.
2. The gradient points in the direction of steepest **increase** of the loss, so to **decrease** the loss we move in the **negative** gradient direction. (1-D sketch: a U-shaped loss curve; current W on one side; arrow `g` pointing uphill; arrow `−g` pointing downhill toward the minimum; updated W one small step downhill.)
3. η controls the **step size** — how far we move along the negative gradient each update. Too small → slow; too large → overshoot/diverge. Update: W ← 2.0 − 0.1×0.5 = 2.0 − 0.05 = **1.95**.
4. W ← 2.0 − 0.1×(−0.5) = 2.0 + 0.05 = **2.05**. A negative gradient means the loss decreases as W increases, so the update moves W **up**; the sign of g flips the direction of the step (−ηg).

中文解析

- **梯度 ≠ 学习**：梯度只是当前点损失上升最快的方向；学习 = 真正**执行更新**并反复迭代。
- **负梯度**：梯度指向损失**上升**最快方向，要**下降**就走**负梯度**方向。
- η = **步长**：太小慢、太大震荡/发散。① g=0.5 → W=**1.95**；② g=−0.5 → W=**2.05**。
- 符号决定方向：更新量 = −ηg，g 变号则移动方向反转。

考点 5.2　Full-Batch GD vs Mini-Batch SGD + Batch Size（概念 + 画图）

文件：`session-5/lecture-3-from-gd-to-sgd.md`；题库 `BoQ-5-ac`（full vs mini）、`BoQ-5-ab`（batch size 权衡）

英文原文（BoQ-5-ac，完整）

> ## Question: Full-Batch Gradient Descent versus Mini-Batch SGD
>
> For a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$, full-batch gradient descent computes a gradient using all n training examples. Mini-batch SGD computes a gradient using a mini-batch $\mathcal{B}$ of size B, where $1 < B \ll n$ in the practical mini-batch meaning of SGD.
>
> Full-batch gradient: $g = \frac{1}{n}\sum_{i=1}^{n} \frac{\partial \ell^{(i)}}{\partial W}$. Mini-batch gradient: $g = \frac{1}{B}\sum_{i\in\mathcal{B}} \frac{\partial \ell^{(i)}}{\partial W}$.
>
> 1. Explain the meaning of B and $\mathcal{B}$. What is the batch size for full-batch gradient descent?
> 2. Draw two optimization paths: a smooth full-batch path and a noisier mini-batch path. Why can full-batch gradient descent be expensive for very large datasets?
> 3. Why can mini-batch SGD be faster in practice? Why can mini-batch noise sometimes help optimization?

中文翻译

> ## 题目：全批量 GD vs 小批量 SGD

> 数据集 $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$。全批量 GD 用全部 n 个样本算梯度；mini-batch SGD 用大小为 B 的小批量 $\mathcal{B}$（$1 < B \ll n$）。公式见上。
>
> 1. 解释 B 与 $\mathcal{B}$ 的含义。全批量 GD 的 batch size 是多少?
> 2. 画两条优化路径：平滑的全批量路径 vs 更嘈杂的小批量路径。为何全批量 GD 对超大数据集很昂贵?
> 3. 为何 mini-batch SGD 实际更快？为何小批量噪声有时**有助于**优化?

答案 Answer（BoQ-5-ac)

English（作答）

> 1. $\mathcal{B}$ is the **mini-batch** (the subset of examples used for one update); $B = |\mathcal{B}|$ is the **batch size** (number of examples in it). For full-batch GD the batch size is **B = n** (the whole dataset).
> 2. Sketch: full-batch = a **smooth** curve heading almost straight to the minimum; mini-batch = a **noisy/zig-zag** path that still trends to the minimum. Full-batch is expensive because **every single update** requires a forward+backward pass over **all n examples**; for very large n this is slow and memory-heavy.
> 3. Mini-batch SGD is faster because each update uses only B≪n examples, so it makes **many more updates per unit of computation**. The **noise** acts like a regularizer: it can help **escape sharp local minima / saddle points** and find flatter, better-generalizing solutions.

中文解析

- $\mathcal{B}$=小批量（一次更新用的样本子集），B=批大小；**full-batch 的 batch size = n。**
- 画图：full-batch **平滑**直奔最小值；mini-batch **嘈杂锯齿**但总体下降。
- full-batch 贵：**每次更新**都要对**全部 n 个样本**前向+反向，超大数据慢且占显存。
- mini-batch 快：每步只用 B≪n 个样本 → 同等算力下**更新次数多得多**；噪声像正则，有助于**逃离尖锐局部极小/鞍点。**

英文原文（BoQ-5-ab，完整)

> ## Question: Batch Size Trade-Off

In mini-batch training, the batch size B affects computation time, gradient noise, memory use, and sometimes generalization.

1. What usually happens when the batch size is very small?
2. What usually happens when the batch size is very large? Why does batch size interact with learning rate?
3. Give one reason a practitioner might choose a medium-sized mini-batch instead of using the entire dataset.
4. Draw a simple comparison of noisy small-batch updates and smoother large-batch updates.

中文翻译

题目：Batch Size 权衡

mini-batch 训练中，批大小 B 影响计算时间、梯度噪声、显存、有时还有泛化。

1. batch size **很小**通常会怎样?
2. batch size **很大**通常会怎样？为何 batch size 会与学习率相互作用?
3. 给一个「选中等批量而非整个数据集」的理由。
4. 画图对比：小批量的嘈杂更新 vs 大批量的平滑更新。

答案 Answer（BoQ-5-ab）

English（作答）

1. Very small batch → **noisy gradients**, zig-zag path, unstable training, poor hardware utilization (but the noise can sometimes help generalization and each update is cheap).
2. Very large batch → **smoother, more accurate** gradients and better hardware use, but each update is **expensive** and it can generalize slightly worse. It interacts with the learning rate because a larger, less noisy gradient usually allows (and often needs) a **larger learning rate** to keep making progress.
3. A medium mini-batch gives a **good balance**: reasonably stable gradients + many fast updates + fits in memory — far cheaper than a full-batch pass over the whole dataset.

> 4. Sketch: small-batch = jagged/noisy arrows wandering toward the minimum; large-batch = smooth arrows heading almost directly to the minimum.

中文解析

- **小批量**：梯度噪声大、路径锯齿、不稳定、硬件利用低；但每步便宜、噪声偶尔助泛化。
- **大批量**：梯度更平滑准确、硬件利用高，但每步**贵**、泛化可能略差；噪声小 → 可/需**更大学习率**。
- **中等批量**：稳定性+更新速度+显存的折中，比全量便宜。

考点 5.3　Momentum（公式默写 + 直觉）

文件：`session-5/lecture-4-momentum.md`；题库 `BoQ-5-ae`

英文原文（BoQ-5-ae，完整）

> ## Question: Momentum
>
> Mini-batch SGD can zig-zag in narrow valleys of the loss surface. Momentum adds a velocity term v that averages gradients over time.
>
> 1. Write the momentum update formulas from memory. Explain the meaning of g, v, β, and η.
> 2. What happens to v when gradients point in a consistent direction for many steps? What happens to v when gradients alternate directions because of oscillation?
> 3. Explain why momentum can accelerate movement along a shallow but consistent direction. What is a common value for β?

中文翻译

> ## 题目：Momentum 动量
>
> mini-batch SGD 在狭窄山谷里会锯齿震荡。动量加入速度项 v，对梯度做时间平均。

1. **默写**动量更新公式。解释 g, v, β, η 的含义。
2. 梯度连续多步方向一致时 v 会怎样？因震荡而方向交替时 v 会怎样？
3. 解释动量为何能在「平缓但一致」的方向上加速。β 常取多少？

答案 Answer（BoQ-5-ae）

English（作答：必默写）

1. v ← β·v + (1−β)·g ; W ← W − η·v.
   - g = mini-batch gradient; v = accumulated **velocity** (smoothed gradient); β = momentum coefficient (how much past gradients carry over); η = learning rate.
2. Consistent direction for many steps → the (1−β)g terms **add up**, so v **grows large** in that direction (acceleration). Oscillating/alternating gradients → positive and negative contributions **cancel**, so v **stays small** (damped oscillation).
3. Along a shallow but consistent direction the gradient is small but always the same sign, so velocity **accumulates** step after step and the effective step becomes larger than plain SGD → faster progress. Common β = **0.9**.

中文解析

- 公式：`v ← βv + (1−β)g`，`W ← W − ηv`（v=速度=平滑后的梯度）。
- 一致方向 → (1−β)g 累加 → v **变大**（加速）；震荡方向 → 正负**抵消** → v **变小**（消震）。
- 平缓一致方向：梯度小但同号 → 速度逐步**累积** → 步子比纯 SGD 大 → 更快。β 常取 **0.9**。

考点 5.4　Adam（一阶/二阶矩 + bias correction + 更新 + 默认值）⭐⭐

文件：`session-5/lecture-5-adam.md`；题库 `BoQ-5-af`（全套公式）、`BoQ-5-ad`（bias correction 专题）

英文原文（BoQ-5-af，完整）

Question: Adam Formulas — First Moment, Second Moment, Update

Adam adapts gradient descent by tracking two moving averages for a gradient g: a first moment m and a second moment v. Use the update notation with learning rate η.

1. Write the first moment update formula. Write the second moment update formula.
2. Explain why Adam's m is similar to momentum. Explain why Adam's v is not the same as the momentum velocity v.
3. Write the bias-corrected first moment formula $\hat{m}$. Write the bias-corrected second moment formula $\hat{v}$.
4. Write the Adam parameter update rule for W. Explain the role of ϵ in the Adam update. List typical default values for η, β_1, β_2, and ϵ.

中文翻译

题目：Adam 公式——一阶矩、二阶矩、更新

Adam 对梯度 g 跟踪两个移动平均：一阶矩 m、二阶矩 v。学习率 η。

1. 写一阶矩、二阶矩更新公式。
2. 为何 Adam 的 m 类似动量？为何 Adam 的 v 与动量的速度 v 不同?
3. 写偏差校正后的 $\hat{m}$、$\hat{v}$。
4. 写 W 的 Adam 更新规则。解释 ϵ 的作用。列出 $\eta, \beta_1, \beta_2, \epsilon$ 的典型默认值。

答案 Answer（BoQ-5-af)

English（作答：必默写）

1. First moment: m ← β_1·m + (1−β_1)·g. Second moment: v ← β_2·v + (1−β_2)·g^2 (element-wise square).
2. m is an exponential moving average of the gradient — exactly like momentum's smoothed velocity, so it captures the **directional trend**. v is **different**: it averages the

squared gradients (magnitude/volatility), not the gradient itself, and is used to **scale** the step per parameter, not to give direction.

3. $\hat{m} = m / (1 - \beta_1^t)$. $\hat{v} = v / (1 - \beta_2^t)$. (t = time step / update count.)
4. $W \leftarrow W - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$. ε is a small constant that prevents **division by zero** (and damps huge steps when $\hat{v}$ is tiny) — numerical stability. Defaults: **$\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e\text{-}8$**.

中文解析

- 一阶矩 $m \leftarrow \beta_1 m + (1-\beta_1) g$；二阶矩 $v \leftarrow \beta_2 v + (1-\beta_2) g^2$（逐元素平方）。
- m ≈ 动量（梯度的 EMA，给**方向**）；v **不同** = 平方梯度的 EMA（给**幅度/波动**），用于**逐参数缩放步长。**
- 校正：$\hat{m} = m/(1-\beta_1^t)$、$\hat{v} = v/(1-\beta_2^t)$。
- 更新：$W \leftarrow W - \eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$；ε 防除零、保数值稳定。默认 **$\eta$=1e-3, β_1=0.9, β_2=0.999, ε=1e-8。**

英文原文（BoQ-5-ad，完整）

Question: Bias Correction in Adam

Adam initializes the moving averages m and v at zero. Early in training, this can bias the moving averages toward zero. Adam therefore uses bias-corrected moments: $\hat{m} = \frac{m}{1-\beta_1^t}$, $\hat{v} = \frac{v}{1-\beta_2^t}$.

1. Why are m and v biased toward zero at the beginning of training?
2. What is the meaning of the time step t? Explain why the correction factor is especially important for small t.
3. What happens to $1 - \beta_1^t$ as t becomes large? What happens to $1 - \beta_2^t$ as t becomes large?
4. Write the full Adam update rule using $\hat{m}$, $\hat{v}$, η, and ϵ.

中文翻译

题目：Adam 的偏差校正

Adam 把 m, v 初始化为 0，训练初期会把移动平均偏向零，故用校正 $\hat{m} = \frac{m}{1-\beta_1^t}$、$\hat{v} = \frac{v}{1-\beta_2^t}$。

1. 为何初期 m, v 偏向零?
2. 时间步 t 的含义？为何 t 小时校正尤其重要?
3. t 变大时 $1-\beta_1^t$、$1-\beta_2^t$ 各趋于什么?
4. 用 $\hat{m}, \hat{v}, \eta, \epsilon$ 写完整 Adam 更新。

答案 Answer（BoQ-5-ad)

English（作答）

1. They start at 0, and each update only mixes in a small fraction (1−β) of the new gradient while keeping a large fraction β of the old (near-zero) average. With β close to 1, the averages stay **dragged toward zero** for the first few steps — an underestimate.
2. t is the **time step** = the index of the current update (t = 1, 2, 3, …). For small t, β^t is still close to 1, so $1-\beta^t$ is small (e.g. ≈ 0.1) and dividing by it **rescales the moment back up**, undoing the zero-initialization bias. So the correction matters most early.
3. As t → large, $\beta_1^t \to 0$ so $\mathbf{1-\beta_1^t \to 1}$; likewise $\mathbf{1-\beta_2^t \to 1}$. The correction factor fades to 1 (no effect), so $\hat{m} \approx m$ and $\hat{v} \approx v$.
4. $W \leftarrow W - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$, with $\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$.

中文解析

- 初期偏零：m,v 从 0 起，每步只掺入 (1−β) 的新梯度、保留 β 的旧（≈0）平均；β≈1 → 前几步被**拽向 0**（低估）。
- t = 更新步数（1,2,3,…）；t 小 → $\beta^t \approx 1$ → $1-\beta^t$ 小 → 除以它把矩**放大还原**，故早期最需要校正。
- t 大 → $\beta^t \to 0$ → $\mathbf{1-\beta^t \to 1}$ → 校正消失，$\hat{m} \approx m$、$\hat{v} \approx v$。
- 完整更新：$W \leftarrow W - \eta \cdot \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$。

考点 5.5　四算法对比 + 轨迹图 GD / SGD / Momentum / Adam ⭐（综合大题）

文件：`session-5/lecture-3`（轨迹图不错）、`code-my_nn-sgd-momentum-adam.py`；题库 `BoQ-5-ag`

英文原文（BoQ-5-ag，完整）

> ### Question: Comparison of Gradient Optimization Algorithms
>
> Optimization algorithms use gradient information to update parameters and minimize loss. Let W be the weight parameter matrix, $\eta > 0$ be the learning rate, and g be the gradient or mini-batch gradient calculated with respect to W.
>
> 1. Write out the standard mathematical update rule for basic gradient descent. Explain how mini-batch Stochastic Gradient Descent (SGD) changes how the gradient vector g is computed compared to full-batch Gradient Descent.
> 2. Write out the complete mathematical update formulas for **Momentum** optimization using a velocity variable v and friction hyperparameter β. Next, write the formulas for **Adam** optimization, explicitly showing the moving averages of the first moment (m), the second moment (v), their bias corrections, and the final parameter update rule.
> 3. Sketch four separate optimization trajectories (paths) in a two-dimensional parameter space (w_1, w_2) representing the convergence behavior of full-batch GD, mini-batch SGD, Momentum, and Adam. Assuming a massive dataset, state which of these methods produces the smoothest path and which requires the highest computational cost *per single parameter update*.
> 4. Identify which of these four optimization techniques are specifically engineered to mitigate the negative effects of high-variance, noisy gradient steps. Briefly explain the mechanism they use to achieve this stability.

中文翻译

> ### 题目：梯度优化算法对比
>
> W 权重矩阵，$\eta > 0$ 学习率，g 为（全/小批量）梯度。

1. 写基本 GD 更新规则。说明 mini-batch SGD 相比 full-batch GD **改变了 g 的计算方式。**
2. 写 **Momentum**（用速度 v、摩擦系数 β）的完整公式；再写 **Adam**（显式给出一阶矩 m、二阶矩 v、各自 bias correction、最终更新）。
3. 在二维参数空间 (w_1, w_2) 画四条轨迹：full-batch GD、mini-batch SGD、Momentum、Adam。设数据集巨大，指出**哪种路径最平滑、哪种每次参数更新计算成本最高。**
4. 指出四者中哪些是**专门为缓解高方差噪声梯度**而设计的，并简述其稳定机制。

答案 Answer（BoQ-5-ag）

English（作答）

1. Basic GD: **W ← W − η•g**. In full-batch GD, g averages gradients over **all n** examples: g = (1/n)Σᵢ $\partial\ell^{(i)}/\partial$W. Mini-batch SGD computes g over a **small random subset** B≪n: g = (1/B)Σ_{i∈B} $\partial\ell^{(i)}/\partial$W — same update rule, but a **noisier, cheaper** gradient estimate.
2. Momentum: v ← β·v + (1−β)·g ; W ← W − η·v. Adam: m ← β_1·m + (1−β_1)·g ; v ← β_2·v + (1−β_2)·g² ; $\hat{m}$ = m/(1−β_1^t) ; $\hat{v}$ = v/(1−β_2^t) ; W ← W − η·$\hat{m}$/(√$\hat{v}$ + ε).
3. Sketch in (w_1,w_2) toward the minimum: full-batch GD = **smooth** near-direct curve; mini-batch SGD = **noisy zig-zag**; Momentum = smoother than SGD, slightly **overshoots/curves** into the valley; Adam = well-damped, fairly direct path. **Smoothest = full-batch GD**. **Highest cost per single update = full-batch GD** (it processes all n examples for one update on a massive dataset).
4. **Momentum and Adam** are designed to reduce noisy-gradient effects. Mechanism: both keep an **exponential moving average of past gradients** (momentum's velocity v / Adam's first moment m), which **averages out** the random noise so the update follows the consistent underlying direction. Adam additionally divides by √$\hat{v}$ (second moment) to **scale down** steps in high-variance directions, adding further stability.

中文解析

- GD：`W ← W − ηg`。full-batch 用**全部 n** 个样本算 g；SGD 用**随机小子集** B≪n → 同一更新式、g 更**嘈杂但便宜。**
- Momentum / Adam 公式见上（与 5.3、5.4 一致）。
- 轨迹：full-batch **平滑直**；SGD **锯齿**；Momentum 比 SGD 平滑、略**冲过头**再拐入谷；Adam 阻尼好、较直。**最平滑 = full-batch GD**；**单次更新成本最高 = full-batch GD**（巨大数据集每步要过全部样本）。

- **抗噪声 = Momentum 与 Adam**：都用**梯度的指数移动平均**把随机噪声**平均掉**，沿一致方向前进；Adam 再除以 $\sqrt{\hat{v}}$ 把高方差方向的步长**缩小**，更稳。

Session 5 速记卡（默写清单）

算法	更新公式（必默写）	关键常数
GD / SGD	$W \leftarrow W - \eta g$	full-batch g 用全部 n；SGD 用 mini-batch B
Momentum	$v \leftarrow \beta v + (1-\beta)g$；$W \leftarrow W - \eta v$	$\beta = 0.9$
Adam	$m \leftarrow \beta_1 m+(1-\beta_1)g$；$v \leftarrow \beta_2 v+(1-\beta_2)g^2$；$\hat{m}=m/(1-\beta_1^t)$；$\hat{v}=v/(1-\beta_2^t)$；$W \leftarrow W - \eta\cdot\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$	η=1e-3, β_1=0.9, β_2=0.999, ε=1e-8

- 画图要会：GD 平滑 / SGD 锯齿 / Momentum 消震加速 / Adam 自适应阻尼。
- 计算题模板：$W_{new} = W - \eta\cdot g$（注意 g 的符号）。
- 核心论断：**Adam = mini-batch SGD + 动量(一阶矩) + 自适应步长(二阶矩) + 偏差校正**；本课程默认优化器。

ML01 期末复习 · 详细整理 —— Session 6 模型选择与正则化 Model Selection & Regularization

> 格式：文件索引/位置 → **英文原文（完整）+ 中文翻译 → 答案（English 作答 + 中文解析）**。
>
> **范围已对齐 xlsx 总表**：session 6 确认考点 = ① **L1/L2 正则（公式+几何画图）** ② **bias-variance（画图）**。 题库映射（正文）：6-ah（L1/L2 公式+几何） | 6-ai（L1/L2 公式） | 6-af（bias-variance 画图）。 其余 BoQ（6-aa 泛化 / 6-ab 分类指标 / 6-ak 协方差）**不在 xlsx 确认范围**，移至文末「附录·超纲备查」。

考点 6.1　L1 / L2 正则化（⭐⭐ 公式 + 几何 + 稀疏）

文件：`session-6/lecture-6-L1-L2-regularization-formulation.md`、`lecture-8-…geometry-and-effect.md`、`lecture-9-L1-L2-first-touch-geometry.md`；题库 `BoQ-6-ah`（完整）、`BoQ-6-ai`（公式）

英文原文（BoQ-6-ah，完整）

> ## Question: L1 and L2 Regularization — Formula, Geometry, and Intuition
>
> In linear regression, regularization helps prevent overfitting by adding penalty terms to the loss function. Let $\mathcal{L}_{train}(W)$ be the original data loss and $\lambda > 0$ be the regularization strength.
>
> 1. Write the complete objective function for linear regression with L1 regularization (Lasso).
> 2. Write the complete objective function for linear regression with L2 regularization (Ridge).
> 3. Which regularization method (L1 or L2) can produce exactly zero weights? Which one tends to shrink weights smoothly?

4. In 2D space with weights w_1 and w_2, draw two figures: (First) data loss contours (ellipses) and the L1 constraint region — mark where a contour first touches the constraint region; (Second) data loss contours (ellipses) and the L2 constraint region — mark where a contour first touches.
5. Based on your drawings, explain why one method leads to sparse solutions (feature selection) while the other does not.
6. Why is λ considered a hyperparameter that needs tuning?

中文翻译

题目：L1/L2 正则——公式、几何、直觉

线性回归中，正则通过给损失加惩罚项防止过拟合。$\mathcal{L}_{train}(W)$ 数据损失，$\lambda > 0$ 正则强度。

1. 写 L1（Lasso）线性回归的完整目标函数。
2. 写 L2（Ridge）的完整目标函数。
3. 哪个能产生**恰好为零**的权重？哪个倾向**平滑收缩**？
4. 2D（w_1, w_2）画两图：损失等高线（椭圆）+ L1 约束区，标**首次相切**点；再画 + L2 约束区，标首次相切点。
5. 据图解释为什么一个产生**稀疏解（特征选择）**而另一个不会。
6. 为何 λ 是需要调的超参数？

答案 Answer（BoQ-6-ah）

English（作答）

1. L1 (Lasso): min_W $\mathcal{L}$_train(W) + λ·Σ$_j$|W$_j$|.
2. L2 (Ridge): min_W $\mathcal{L}$_train(W) + λ·Σ$_j$ W$_j$².
3. **L1** can produce exactly zero weights (sparsity); **L2** shrinks weights smoothly toward zero (rarely exactly zero).
4. Sketch: elliptical loss contours centered at the unconstrained optimum W*. L1 region = a **diamond** (|w$_1$|+|w$_2$|≤c) with corners on the axes at (±c,0),(0,±c); the expanding ellipse usually **first touches a corner** → one weight = 0. L2 region = a **circle** (w$_1$²+w$_2$²≤c); the ellipse first touches at a **smooth point** off the axes → both weights nonzero.

5. L1's diamond has sharp **corners on the coordinate axes**, so the first-touch point is typically a corner where a coordinate is exactly 0 → sparse / feature selection. L2's circle is smooth with no corners, so contact happens at a generic point with all weights nonzero → no sparsity, just shrinkage.
6. λ trades off data fit vs simplicity: λ=0 → no regularization (overfit risk); λ→∞ → weights forced toward 0 (underfit). The best λ is data-dependent and is tuned on a validation set.

中文解析

- L1：`min ℒ_train + λΣ\|Wⱼ\|`；L2：`min ℒ_train + λΣWⱼ²`。
- **L1 出零（稀疏）**，L2 平滑收缩。
- 画图：椭圆等高线 + L1**菱形**（顶点在轴上）→ 首次相切多在**顶点**→某权重=0；L2**圆**→相切在光滑点→权重都非零。
- 稀疏原因：菱形**顶点落在坐标轴**上 → 触点处某坐标恰为 0；圆无顶点 → 不稀疏。
- λ 是数据拟合 vs 简单度的权衡：0→过拟合、∞→欠拟合，需在验证集调。

英文原文（BoQ-6-ai，完整）

Question: L1 and L2 Regularization Formulas

Regularization adds a penalty to the loss to discourage overly complex models. Let W_j denote a weight parameter and λ denote regularization strength.

1. Write a regularized objective using an empirical loss $\mathcal{L}_{train}(W)$ plus an L1 penalty.
2. Write a regularized objective using $\mathcal{L}_{train}(W)$ plus an L2 penalty. Explain the role of λ.
3. What happens when $\lambda = 0$? What can happen when λ is extremely large?
4. Explain why regularization can improve validation performance even if it increases training loss.

中文翻译

题目：L1/L2 正则公式

1. 写 $\mathcal{L}_{train}$ + L1 惩罚的目标。
2. 写 + L2 惩罚的目标，解释 λ 的作用。
3. $\lambda = 0$ 会怎样？λ 极大会怎样？
4. 为何正则即使**提高训练损失**也能改善验证表现？

答案 Answer（BoQ-6-ai）

English（作答）

1. L1: $\mathcal{L}$_train(W) + λ·Σ$_j$|W$_j$|.
2. L2: $\mathcal{L}$_train(W) + λ·Σ$_j$ W$_j$². λ controls regularization strength — how strongly large weights are penalized relative to fitting the data.
3. λ=0 → no penalty, pure data fitting → can overfit. λ extremely large → penalty dominates, weights pushed toward 0 → underfitting (model too simple).
4. Regularization restricts the effective capacity, so the model fits training data slightly worse (higher train loss) but learns a **simpler, smoother** function that generalizes better → lower validation loss.

中文解析

- L1=`ℒ+λΣ\|Wⱼ\|`、L2=`ℒ+λΣWⱼ²`；λ=惩罚强度。
- λ=0→纯拟合可能过拟合；λ→∞→权重压到 0 欠拟合。
- 正则牺牲一点训练损失换**更简单泛化更好**的解 → 验证更好。

考点 6.2　Bias–Variance 权衡（⭐ 画图）

文件：`session-6/lecture-5-model-selection-bias-variance.md`；题库 `BoQ-6-af`

英文原文（BoQ-6-af，完整）

Question: Bias–Variance Tradeoff Drawing

The bias–variance tradeoff describes how model complexity affects underfitting and overfitting.

1. Explain high bias in words. Explain high variance in words.
2. Draw training error and validation error as model complexity increases. Mark the underfitting region. Mark the overfitting region. Mark a reasonable model-complexity region.
3. Give one example of a model that might have high bias. Give one example of a model that might have high variance.

中文翻译

题目：偏差-方差权衡画图

1. 用文字解释**高偏差**、**高方差**。
2. 画训练误差与验证误差随**模型复杂度**增大的曲线。标出欠拟合区、过拟合区、合理复杂度区。
3. 各举一个高偏差、高方差模型的例子。

答案 Answer（BoQ-6-af）

English（作答）

1. **High bias** = the model is too simple to capture the underlying pattern → underfits → high error on **both** train and validation. **High variance** = the model is too complex and fits noise in the training data → low train error but high validation error (large gap).
2. Sketch: x-axis = model complexity, y-axis = error. **Training error** decreases monotonically. **Validation error** is U-shaped (high → down → up). Left region (both high) = **underfitting / high bias**; right region (train low, val rising) = **overfitting / high variance**; the **bottom of the validation U** = reasonable complexity (sweet spot).
3. High bias example: linear regression / logistic regression on a non-linear problem. High variance example: a very deep/wide neural network (or high-degree polynomial) on a small dataset.

中文解析

- 高偏差=太简单→欠拟合→train/val 都高；高方差=太复杂拟合噪声→train 低 val 高（gap 大）。

- 图：train 误差单调降；val 误差 **U 形**；左=欠拟合(高偏差)、右=过拟合(高方差)、**U 底=sweet spot。**
- 例：线性/逻辑回归=高偏差；深大网络/高次多项式在小数据=高方差。

Session 6 速记（xlsx 确认范围）

考点	必记
L1	$\mathcal{L}+\lambda\Sigma\|w_j\|$；**菱形**；顶点在轴 → **稀疏/特征选择**
L2	$\mathcal{L}+\lambda\Sigma w_j^2$（weight decay）；**圆**；平滑收缩、不稀疏
λ	0→过拟合、∞→欠拟合；验证集调
bias-variance	train 单调降、val **U 形**；左欠拟合(高偏差)、右过拟合(高方差)、U 底 sweet spot
不考	transfer learning

附录 • 超出 xlsx 确认范围（仅备查，⚠️ 老师确认范围未列）

> 以下 3 题是真实存在的 session-6 BoQ，但**不在 xlsx 总表的 session6 考点行**内。按「对齐 xlsx」要求移到此处，仅作备查，复习时可低优先级或跳过。

附 A1　泛化、过拟合与数据划分（BoQ-6-aa）

英文原文（完整）

> ### Question: Generalization, Overfitting, and Data Splits
>
> A classifier achieves 99% accuracy on its training set but only 60% accuracy on a validation set sampled from the same data distribution.

1. Define the difference between training performance and generalization performance. What does the 39% performance gap suggest about the model's current state?
2. Explain why a model that achieves a very low training loss or near-perfect training accuracy can still be a poor model. Why is memorizing training examples fundamentally different from learning a useful underlying pattern?
3. Sketch a single plot illustrating an overfitting model over training epochs (training loss & validation loss curves).
4. Mark where the model begins to overfit. Explain why a validation dataset is essential, and why validation data must never be used to update parameters during gradient descent.
5. Provide two distinct regularization or data-driven strategies to narrow this generalization gap.

答案（要点）

1. 训练表现 = 训练集上的精度/损失；泛化表现 = **未见数据**上的表现。39% gap（99%→60%）= **严重过拟合。**
2. 低训练损失可能只是**背下样本（含噪声）**，不迁移；学模式=抓底层结构。
3. 图：train 单调降、val 先降后**升。**
4. 过拟合点 = val **最低点之后开始上升**处；val 只能用于评估，若更新参数会泄漏、失去无偏性。
5. ① L1/L2 或 dropout；② 更多数据/数据增强（含 early stopping、减小容量）。

附 A2　分类指标与类别不平衡（BoQ-6-ab）

英文原文（完整）

Question: Metrics Under Class Imbalance and Task-Dependent Trade-offs

A binary medical screening classifier: TP = 40, FN = 10, FP = 160, TN = 790. Accuracy=(TP+TN)/total, Precision=TP/(TP+FP), Recall=TP/(TP+FN), F1=2·(P·R)/(P+R).

1. Compute accuracy, precision, recall, F1 (show calculations).
2. Why can accuracy alone mislead on imbalanced data here?

3. Why is recall often prioritized in screening? Interpret the 10 false negatives.
4. In a spam-filter scenario, why may precision become primary? Interpret many false positives.
5. Propose one threshold-tuning strategy per domain.

答案（含计算）

1. Acc=(40+790)/1000=**0.83**；P=40/200=**0.20**；R=40/50=**0.80**；F1=2·(0.2·0.8)/(0.2+0.8)=**0.32**。
2. 95% 为负类 → 多猜「负」也高 accuracy；0.83 掩盖正类差（P 仅 0.20）。
3. 医学：**假阴性致命** → 优先 recall；10 FN = 10 个病人被漏诊。
4. 垃圾邮件：**假阳性**=正常邮件被误删 → 优先 precision。
5. 降阈值↑recall（医学）、升阈值↑precision（垃圾邮件），验证集/PR 曲线上调。

附 A3　协方差矩阵与二元正态（BoQ-6-ak）

英文原文（完整）

Question: Visualizing and Analyzing a Bivariate Normal Distribution

`sigma=[[1.0,0.6],[0.6,1.0]]`, `np.random.multivariate_normal(mu, sigma, 1000)`, scatter with `axis('equal')`.

1. Name of matrix `sigma`? What do diagonal (1.0) and off-diagonal (0.6) elements represent about x and y?
2. Sketch the scatter; describe shape/orientation; how does 0.6 set the slope/tilt?
3. Draw three scatter sketches for off-diagonal = −0.8, 0, 0.8.

答案（要点）

1. `sigma`=**协方差矩阵**；对角=各自**方差**(1.0)；非对角 0.6=**协方差**→x,y **正相关**。
2. 散点=**倾斜椭圆云**，沿 **y=x**（约 45°）；0.6 越大越细长。

3. −0.8→沿 **y=−x**（负斜率）； 0→**圆形**（无相关）； 0.8→沿 y=x **强细长。**

ML01 期末复习 · 详细整理 —— Session 7 正则化技术 Regularization Techniques

> 格式：文件索引/位置 → **英文原文（完整）+ 中文翻译 → 答案（English 作答 + 中文解析）**。
>
> **范围已对齐 xlsx 总表**：session 7 确认考点 = **dropout（概念+inverted dropout 公式/期望+填空）、early stopping（画图）、data augmentation（了解即可）**；transfer learning **不考**。
>
> 题库映射（正文）：7-aa（记忆过拟合 + early stopping，**一文两题**）｜7-ab（dropout 子网络）｜7-ac（inverted dropout 期望）｜7-ah（dropout 填空）｜7-ae（data augmentation）。 其余 BoQ（7-af 超参失效模式）**跨 session、不在 xlsx 确认范围**，移至文末「附录·超纲备查」。

考点 7.1　Dropout（概念 + 训练/推理差异）

文件：`session-7/lecture-1-dropout.md`；题库 `BoQ-7-ab`

英文原文（BoQ-7-ab，完整）

> ## Question: Dropout as Training Many Sub-Networks
>
> Dropout randomly removes some activations during training. With drop probability p, each mini-batch may effectively train a slightly different sub-network.
>
> 1. Draw a neural network layer before and after dropout is applied.
> 2. Explain in words what dropout does during training. Why can dropout reduce co-adaptation between neurons, and why can this improve generalization?
> 3. What does dropout do during evaluation/inference?
> 4. Why should dropout behavior be different during training and evaluation?

中文翻译

题目：dropout 作为训练众多子网络

dropout 训练时随机移除部分激活，丢弃概率 p，每个 mini-batch 实际训练一个略不同的子网络。

1. 画 dropout 前后的网络层。
2. 文字说明 dropout 训练时做什么。为何能减少神经元**共适应(co-adaptation)**、为何改善泛化?
3. **评估/推理**时 dropout 做什么?
4. 为何训练与评估时行为应不同?

答案 Answer（BoQ-7-ab)

English（作答）

1. Sketch: "before" = a fully-connected layer with all neurons active; "after" = the same layer with a random subset of neurons crossed out (their outputs set to 0), and only the surviving connections drawn.
2. During training, dropout randomly zeroes each activation with probability p, so each step trains a different **sub-network**. Neurons cannot rely on specific other neurons always being present, which **reduces co-adaptation**; each neuron must learn features useful on its own. This acts like training/averaging an **ensemble** of many sub-networks → better generalization.
3. During evaluation/inference, dropout is **turned off**: all neurons are used (no random dropping). (With inverted dropout, no extra rescaling is needed because scaling was done in training.)
4. Training needs randomness to regularize and create sub-networks; inference must be **deterministic and use the full network** to give a stable prediction. Using the whole network at test time approximates averaging over all sub-networks.

中文解析

- 画图：前=全连接全激活；后=随机一部分神经元被划掉（输出置 0）。
- 训练：以概率 p 随机置零 → 每步训练不同**子网络** → 神经元不能依赖特定伙伴 → **减少共适应** → 类似**集成**多个子网络 → 泛化更好。

- 推理：**关掉 dropout**，用全部神经元（inverted dropout 无需再缩放）。
- 差异原因：训练靠随机性正则；推理要**确定 + 用整网**给稳定预测（≈对所有子网络平均）。

考点 7.2　Inverted Dropout（⭐ 公式 + 期望证明 + 代码填空）

文件：`session-7/lecture-2-dropout-1-p-multipy-or-devide.md`、`code-my_nn-dropout-L1-L2.py`；题库 `BoQ-7-ac`（期望）、`BoQ-7-ah`（填空）

英文原文（BoQ-7-ac，完整）

> ## Question: Inverted Dropout Formula and Expectation
>
> Inverted dropout randomly drops activations during training and scales the surviving activations so that no extra scaling is needed during inference. Let p be the drop probability. For one activation a_i, sample a keep mask $m_i \sim \text{Bernoulli}(1-p)$ ($m_i = 1$ keep, $m_i = 0$ drop). Output: $\tilde{a}_i = a_i \frac{m_i}{1-p}$.
>
> 1. Draw a small layer before and after dropout, showing kept vs dropped units.
> 2. Explain why inverted dropout divides by $1-p$ during training.
> 3. Show the expectation calculation proving that $\mathbb{E}[\tilde{a}_i] = a_i$.
> 4. State what happens to the dropout layer during evaluation/inference.

中文翻译

> ## 题目：inverted dropout 公式与期望
>
> 丢弃概率 p，keep mask $m_i \sim \text{Bernoulli}(1-p)$，输出 $\tilde{a}_i = a_i \frac{m_i}{1-p}$。
>
> 1. 画 dropout 前后的小网络层（标保留/丢弃）。
> 2. 解释训练时为何**除以** $1-p$。
> 3. 证明 $\mathbb{E}[\tilde{a}_i] = a_i$。
> 4. 评估/推理时 dropout 层会怎样?

答案 Answer（BoQ-7-ac）

English（作答）

> 1. Sketch: before = all units active; after = some units dropped (output 0), surviving units' outputs scaled up by 1/(1−p).
> 2. Dropping keeps only a fraction (1−p) of activations, which would **shrink the expected total signal** by (1−p). Dividing the survivors by (1−p) **restores the expected magnitude**, so the layer's output distribution matches what inference (full network) will see → no test-time rescaling needed.
> 3. E[ã_i] = E[a_i · m_i/(1−p)] = a_i/(1−p) · E[m_i] = a_i/(1−p) · (1−p) = **a_i**. (since E[m_i] = P(keep) = 1−p.)
> 4. During inference the dropout layer is **inactive**: it passes activations through unchanged (no dropping, no scaling), because the 1/(1−p) scaling was already applied in training.

中文解析

- 除以 (1−p) 的原因：只留下 (1−p) 比例的激活 → 期望幅度缩小 (1−p) 倍 → 除以 (1−p) **补偿还原**，使训练/推理分布一致，推理无需再缩放。
- 期望证明：`E[ã_i] = a_i/(1-p)·E[m_i] = a_i/(1-p)·(1-p) = a_i`（因 `E[m_i]=1-p`）。
- 推理：dropout **失效**，原样透传。

英文原文（BoQ-7-ah，填空题）

> ## Question: Dropout Fill-in-the-Blank
>
> ```
> self.mask = np.random.binomial(1, 1 - p, size=input.shape) / _____BLANK_1_____
> output = input * self.mask
> ```
>
> If we only did `mask = np.random.binomial(1, 1-p, ...)`, then on average only a fraction `_BLANK_2_` of activations would be alive, so the expected activation magnitude would shrink by a factor of `_BLANK_3_`. To compensate, inverted dropout scales up survivors by `_BLANK_4_` during training. When we set `self.training = _BLANK_5_`, the dropout layer returns the raw input activations.

答案 Answer（BoQ-7-ah）

> - `BLANK_1`（除以）：`1 - p`
> - `BLANK_2`（存活比例）：`1 - p`
> - `BLANK_3`（缩小因子）：`1 - p`
> - `BLANK_4`（放大倍数）：`1 / (1 - p)`
> - `BLANK_5`（推理时）：`False`

中文解析：训练用 `binomial(1,1-p)/(1-p)` 生成并放大 mask；只乘 mask 会使期望缩小到 (1−p)，故放大 1/(1−p)；`self.training=False` 时返回原始激活（关闭 dropout）。

考点 7.3　Early Stopping（⭐ 画图 + patience + 正则解释）

文件：`session-7/lecture-3-early-stopping.md`；题库 `BoQ-7-aa`（第二题）

英文原文（BoQ-7-aa 第二题，完整）

> ## Question: Early Stopping with Validation Loss
>
> Early stopping stops training when validation performance no longer improves, even if training loss continues decreasing.
>
> 1. Draw a validation-loss curve where early stopping would be useful. Mark the epoch with the best validation loss.
> 2. Explain the idea of "patience" in early stopping.
> 3. Why might stopping at the lowest training loss be a bad idea?
> 4. Explain how early stopping acts like a regularization method.

中文翻译

> ## 题目：基于验证损失的 early stopping
>
> 1. 画一条适合 early stopping 的验证损失曲线，标出**最佳验证损失**所在 epoch。

> 2. 解释 early stopping 里 **patience（耐心）** 的概念。
> 3. 为何「停在最低训练损失」是坏主意?
> 4. 解释 early stopping 为何像一种正则化。

答案 Answer（BoQ-7-aa 第二题）

English（作答）

> 1. Sketch: x = epochs, y = validation loss; a **U-shaped** curve that decreases, reaches a minimum at t^*, then rises. Mark the lowest point $t^* = \arg\min_t \mathcal{L}_{val}(t)$ — that is where we stop.
> 2. **Patience** = the number of epochs we allow validation loss to **not improve** before stopping. If val loss fails to beat the best for k consecutive epochs, training stops. This tolerates small noisy fluctuations instead of stopping at the first uptick.
> 3. Training loss decreases monotonically, so its lowest point is at the very end — exactly where the model has **overfit** the most. Stopping there gives the worst generalization; we want the validation minimum, not the training minimum.
> 4. Early stopping limits how long the model optimizes, which **limits its effective capacity** (it can't fully fit the noise). This is implicit regularization: it selects parameters from before the overfitting region → smoother function, better generalization.

中文解析

- 图：验证损失 **U 形**，标最低点 `t*=argmin L_val(t)`（=停的位置）。
- **patience**：允许验证损失连续 k 个 epoch **不改善**才停，容忍小波动。
- 停在最低**训练**损失=训练末端=过拟合最严重处 → 泛化最差；要的是**验证**最低点。
- 像正则：限制优化时长→限制**有效容量**→停在过拟合前→更平滑、泛化更好。

考点 7.4　记忆过拟合的症状与对策（概念 + 画图）

文件：`session-7/lecture-1-dropout.md`、`lecture-3-early-stopping.md`；题库 `BoQ-7-aa`（第一题）

英文原文（BoQ-7-aa 第一题，完整）

Question: A Neural Network That Memorizes Too Well

You train a neural network for image classification. After many epochs, training accuracy is almost 100%, but validation accuracy is much lower.

1. What does this symptom suggest?
2. Draw training and validation accuracy curves for this situation. Draw also the training and validation loss curves.
3. Explain why increasing model capacity can make this problem worse on a small dataset.
4. Propose two regularization or data strategies that could improve validation performance.

中文翻译

题目：记忆能力过强的网络

训练精度近 100%，验证精度低很多。

1. 这症状说明什么?
2. 画训练/验证的**准确率**曲线，以及训练/验证的**损失**曲线。
3. 为何在**小数据集**上增大模型容量会让问题更糟?
4. 给两个改善验证表现的正则/数据策略。

答案 Answer（BoQ-7-aa 第一题)

English（作答）

1. **Overfitting**: the model memorized the training data but fails to generalize (large train–val gap).
2. Accuracy: training accuracy rises toward ~100%; validation accuracy rises then **plateaus/drops**, staying well below training. Loss: training loss keeps decreasing; validation loss decreases then **turns upward** (the gap = overfitting).
3. On a small dataset there are few examples, so a high-capacity model has more than enough parameters to **memorize every example (including noise)** instead of

learning general patterns → the train–val gap widens.
4. (a) Add regularization: dropout and/or L2 (and early stopping). (b) Get more/augmented data (data augmentation) — also reducing model capacity helps.

中文解析

- 症状=**过拟合**（记住训练集、不泛化、gap 大）。
- 准确率：train→100%、val 升后**停滞/下降**；损失：train 单调降、val 先降后**升**。
- 小数据+大容量 → 参数足以**背下每个样本（含噪声）** → gap 更大。
- 对策：① dropout/L2/early stopping；② 更多数据/数据增强（减小容量也行）。

考点 7.5　Data Augmentation（了解即可）

文件：`session-7/lecture-4-data-augmentation.md`；题库 `BoQ-7-ae`

英文原文（BoQ-7-ae，完整）

Question: Data Augmentation as Label-Preserving Transformation

Data augmentation creates new training examples by applying transformations that should not change the label (e.g. a small shift of an MNIST digit keeps the same label).

1. Define data augmentation and explain how it modifies the effective capacity and generalization behavior of a model.
2. Provide three distinct examples of label-preserving transformations commonly used for computer vision.
3. Describe the primary challenges of designing data augmentation for NLP (text) compared to vision. Give one example of a text-based transformation.

中文翻译

题目：data augmentation 作为保标签变换

1. 定义数据增强，说明它如何改变**有效容量**与泛化。
2. 给三个 CV 常用的**保标签**变换。
3. 与图像相比，NLP 文本做数据增强的主要难点？给一个文本变换例子。

答案 Answer（BoQ-7-ae）

English（作答）

1. Data augmentation = generating new training examples by applying **label-preserving transformations** to existing data. It effectively **increases the training set size / diversity**, lowering variance and improving generalization (acts as regularization) without collecting new labels.
2. Vision examples: random crop/shift, horizontal flip, rotation (also scaling, brightness/contrast jitter, adding noise).
3. Text is **discrete and order/grammar-sensitive**, so small edits can easily **change meaning or break the label** (unlike a shifted image). It is hard to transform while preserving semantics. Example text transformation: **synonym replacement** (or back-translation).

中文解析

- 定义=对已有数据做**保标签变换**生成新样本 → 等效**扩大数据量/多样性** → 降方差、改善泛化（正则作用）。
- CV 例：随机裁剪/平移、水平翻转、旋转（还有缩放、亮度抖动、加噪）。
- NLP 难点：文本**离散、对语序/语法敏感**，小改动易**改变语义/破坏标签**；例：**同义词替换**（或回译 back-translation）。

Session 7 速记（xlsx 确认范围）

考点	必记
dropout 概念	训练随机置零→子网络/集成、减共适应；推理关掉用整网

考点	必记
inverted dropout	`ã = a·m/(1−p)`，`E[ã]=a`；填空：1-p,1-p,1-p,1/(1-p),False
early stopping	验证损失 U 形，停 `t*=argmin L_val`；patience=连续 k 轮不改善才停；隐式正则
data augmentation	保标签变换→扩数据；CV：裁剪/翻转/旋转；NLP 难（离散语义）→同义词替换
不考	transfer learning

附录 · 超出 xlsx 确认范围（仅备查，⚠️ 老师确认范围未列）

> 7-af 是真实存在的 session-7 BoQ，但它**跨 session**（涉及 η、λ、batch size、容量、dropout p），不在 xlsx 总表的 session7 考点行内。按「对齐 xlsx」移到此处，仅作备查。

附 B1　超参数失效模式（BoQ-7-af）

英文原文（完整）

> ## Question: Hyperparameter Configurations and Failure Modes
>
> Describe the typical impact on both training and validation performance for each:
>
> 1. The learning rate η is excessively large.
> 2. The learning rate η is excessively small (non-zero).
> 3. The dropout drop probability p is too close to 1.
> 4. The L2 penalty strength λ is too close to 0.
> 5. The network depth/capacity is exceptionally large relative to a very small training dataset.
> 6. The mini-batch size is extremely small (e.g. 1).

答案（要点）

1. **η 过大** → 越过极小点、震荡/**发散**(NaN)，train/val 都差。
2. **η 过小** → 训练**极慢**，预算内难收敛 → 欠拟合。
3. **p→1** → 几乎全丢 → 学不动 → **欠拟合**（train/val 损失都高）。
4. **λ→0** → 几乎无正则 → **过拟合**（train 低、val 高、gap 大）。
5. **大容量 + 极小数据** → 记忆 → train≈100%、**val 差**（严重过拟合）。
6. **batch=1** → 梯度**极嘈杂**、训练不稳/锯齿、慢；可能仍泛化但需更小学习率。

注：这些点分别对应 session5（η/batch）、session6（λ/容量）、session7（dropout p），是很好的**跨章综合复习**，但严格按 xlsx 不算 session7 独立考点。

ML01 期末复习 · 详细整理 —— Extra Transformer（201 注意力 / 202 位置编码 / 203 掩码，约占 30%）

> 格式：文件索引/位置 → **英文原文（完整）+ 中文翻译 → 答案（English 作答 + 中文解析）**。
>
> 范围：3 个 transformer extra session，各 1 道 BoQ（`-ee`），记住即可。
>
> 题库映射：201-ee（self vs cross attention）｜202-ee（正弦位置编码）｜203-ee（因果掩码）。

考点 E1（201） Self-Attention vs Cross-Attention（⭐ QKV 公式 + 来源）

文件：`session-201-qkv-attention-mini-series/lecture-7-matrix-form-of-attention.md`、`lecture-16-cross-attention.md`、`lecture-17-self-attention-vs-cross-attention.md`；题库 `BoQ-201-ee`

英文原文（BoQ-201-ee，完整）

> ## Question: Self-Attention vs Cross-Attention
>
> In encoder-decoder transformers (e.g. *Attention is All You Need*), both self-attention and cross-attention are used.
>
> 1. Write the common scaled dot-product attention formula.
> 2. In a standard Transformer self-attention layer, explicitly state where the input vectors for Q, K, and V originate.
> 3. In an encoder-decoder cross-attention layer, explicitly state where the input vectors for the queries Q originate, and where the vectors for the keys K and values V originate.
> 4. Explain why cross-attention is crucial for traditional sequence-to-sequence tasks like machine translation. How does it allow the model to align target tokens with source

tokens?
5. State whether state-of-the-art autoregressive LLMs such as DeepSeek and ChatGPT utilize a decoder-only, encoder-only, or encoder-decoder architecture. Based on your answer, explain whether these models require cross-attention layers during text generation.

中文翻译

题目：自注意力 vs 交叉注意力

编码器-解码器 transformer（《Attention is All You Need》）同时用自注意力与交叉注意力。

1. 写通用的**缩放点积注意力**公式。
2. 标准 transformer **自注意力**层中，Q, K, V 的输入向量各来自哪里？
3. 编码器-解码器**交叉注意力**层中，Q 来自哪里？K, V 来自哪里？
4. 为何交叉注意力对机器翻译等 seq2seq 任务至关重要？它如何让模型把目标 token 与源 token **对齐**？
5. DeepSeek/ChatGPT 这类自回归 LLM 用 decoder-only / encoder-only / encoder-decoder？据此说明它们生成文本时**是否需要交叉注意力**。

答案 Answer（BoQ-201-ee）

English（作答）

1. Attention(Q, K, V) = softmax(QK^T / √d_k) · V. (d_k = key/query dimension; the √d_k scaling keeps the dot products from getting too large.)
2. In **self-attention**, Q, K, and V all come from the **same input sequence** — they are three different linear projections of the **same layer's input representations** (X): Q = XW_Q, K = XW_K, V = XW_V.
3. In **cross-attention** (decoder), the queries **Q come from the decoder** (the target-side representations of the current layer), while the keys **K and values V come from the encoder output** (the source-side representations).
4. Cross-attention lets each target (output) token **look back at all source (input) tokens** and pull in the relevant source information. The softmax attention weights act as a **soft alignment** between target and source positions — e.g. when generating a

translated word, the model attends most to the corresponding source word(s). Without it, the decoder could not access the source sentence.

5. ChatGPT and DeepSeek are **decoder-only** autoregressive models. Because there is no separate encoder, they do **not** use cross-attention — they only use (masked) self-attention over the running sequence of tokens during generation.

中文解析

- 公式：`Attention(Q,K,V)=softmax(QKᵀ/√d_k)·V`（√d_k 防点积过大）。
- **自注意力**：Q,K,V 来自**同一序列**（同层输入 X 的三种线性投影 XW_Q/XW_K/XW_V）。
- **交叉注意力**：**Q 来自解码器**（目标侧），**K,V 来自编码器输出**（源侧）。
- 翻译为何关键：让每个目标 token **回看全部源 token**，softmax 权重=目标-源的**软对齐**（生成某词时主要关注对应源词）。
- LLM：ChatGPT/DeepSeek = **decoder-only** → **无交叉注意力**，生成时只用（带因果掩码的）自注意力。

考点 E2（202） Sinusoidal Positional Encoding（⭐ 公式 + 加法注入）

文件：`session-202-positional-encoding-mini-series/lecture-pe-7-sinusoidal-formulation.md`、`lecture-pe-8-why-addition-works.md`；题库 `BoQ-202-ee`

英文原文（BoQ-202-ee，完整）

> ## Question: Sinusoidal Positional Encoding and the Attention Pipeline
>
> Self-attention is permutation-invariant with respect to token positions. To preserve order, a positional encoding vector is added to each token embedding before the first attention layer. For a token at position i (0-indexed), with d_{model} the embedding dimension and k indexing sine/cosine pairs:

$$PE_{i,2k} = \sin\left(\frac{i}{10000^{\,2k/d_{\text{model}}}}\right), \qquad PE_{i,2k+1} = \cos\left(\frac{i}{10000^{\,2k/d_{\text{model}}}}\right)$$

1. For position $i = 0$, compute the values in the sine channels (even indices) and the cosine channels (odd indices). Explain why the result is the same regardless of d_{model}.
2. In one sentence, explain why using many different frequencies (different k) is helpful for representing position.
3. Why does this sinusoidal method not require learning extra positional parameters?
4. Write the formula that combines token embedding x_i with its positional vector p_i to produce the representation fed into attention.
5. If two tokens are the same word but appear at different positions, why can their combined vectors still differ?
6. After adding positional information, the next step is scaled dot-product attention. Explain in simple words why injecting position *before* attention helps the model with next-token prediction.

中文翻译

题目：正弦位置编码与注意力管线

自注意力对位置**置换不变**，为保留顺序，在第一层注意力前给每个 token embedding **加**位置编码。位置 i（0 起），公式见上。

1. 对 $i = 0$，算正弦通道（偶索引）与余弦通道（奇索引）的值。为何结果与 d_{model} 无关?
2. 一句话：为何用多种频率（不同 k）有助于表示位置?
3. 为何该正弦方法**不需要学习**额外位置参数?
4. 写把 token embedding x_i 与位置向量 p_i 结合、喂入注意力的公式。
5. 同一个词出现在不同位置，为何结合后的向量仍能不同?
6. 位置注入在注意力**之前**，为何这对下一词预测有帮助？（用简单语言）

答案 Answer（BoQ-202-ee）

English（作答）

1. At i = 0: every sine channel = sin(0) = **0**, every cosine channel = cos(0) = **1**. The argument is i / 10000^(2k/d_model) = 0 / (anything) = 0 for **all** k, so the

denominator (which is the only place d_model appears) does not matter → the result is the same for any d_model.
2. Different frequencies let the encoding capture position at **multiple scales** (fast-changing channels distinguish nearby positions, slow-changing channels capture long-range position), giving a unique, distinguishable code for every position.
3. The encodings are **fixed deterministic functions** of position (sin/cos), computed directly — there are no trainable weights to learn.
4. z_i = x_i + p_i = x_i + PE_i (element-wise addition of the positional vector to the token embedding).
5. Even if x_i (the word embedding) is identical, the **added p_i = PE_i differs by position**, so the sums x_i + PE_i differ → the model can tell the two occurrences apart.
6. Attention by itself ignores order (permutation-invariant), so without position it would treat a sentence as a bag of words. Injecting position **before** attention lets the attention scores depend on **where** tokens are, which is essential because next-token prediction depends on word **order** (e.g. "dog bites man" ≠ "man bites dog").

中文解析

- i=0：正弦通道全 = sin(0)=**0**，余弦通道全 = cos(0)=**1**；自变量 = i/10000^(...) = 0/任意 = 0，**d_model 只出现在分母**，故与 d_model 无关。
- 多频率：在**多种尺度**上编码位置（高频分辨近邻、低频抓长程），每个位置都有唯一可区分的编码。
- 不需学习：sin/cos 是位置的**固定确定函数**，直接算出，无可训练参数。
- 结合公式：`z_i = x_i + p_i = x_i + PE_i`（逐元素相加）。
- 同词不同位：x_i 相同但 **p_i 随位置不同** → 和不同 → 可区分。
- 注意力前注入：注意力本身**忽略顺序**（置换不变），注入位置后注意力分数才依赖**位置**；下一词预测依赖**语序**（"dog bites man"≠"man bites dog"）故必需。

考点 E3（203）　Causal Mask 因果掩码（⭐ j≤i + −∞ + 防泄漏）

文件：`session-203-masking-mini-series/lecture-mask-3-causal-mask-autoregressive.md`、`lecture-mask-7-masking-in-transformers.md`；题库 `BoQ-203-ee`

英文原文（BoQ-203-ee，完整）

Question: Causal Mask for Autoregressive Decoding

In next-token generation, each token should use only itself and earlier tokens, not future tokens. A causal mask enforces this inside self-attention.

1. Let i denote the query index and j the key index. State the basic mathematical relationship between i and j that determines whether an attention connection is causally valid.
2. For a sequence of length $n = 4$ (0-based: 0,1,2,3), list the exact key positions j that query position $i = 2$ is permitted to attend to under strict causal masking.
3. Explain why omitting a causal mask during training leads to target data leakage. Why does this leakage make training loss appear exceptionally low while producing broken generation during inference?
4. In implementation, masked positions are set to $-\infty$ before softmax. Why does this effectively block those positions?
5. In one sentence, explain why causal masking is required even when full sequences are available during training.

中文翻译

题目：自回归解码的因果掩码

下一词生成中，每个 token 只能用自己和更早的 token，不能用未来。因果掩码在自注意力里强制此规则。

1. 设 i 查询索引、j 键索引。写决定连接**因果有效**的 i, j 基本关系。
2. 序列长 $n = 4$（0 起：0,1,2,3），严格因果掩码下查询位置 $i = 2$ 允许关注哪些 j?
3. 训练时省略因果掩码为何导致**目标数据泄漏**？为何泄漏使训练损失异常低、但推理生成崩坏？
4. 实现中 masked 位置在 softmax 前设 $-\infty$，为何能有效屏蔽？
5. 一句话：为何训练时即使有完整序列也仍需因果掩码？

答案 Answer（BoQ-203-ee）

English（作答）

> 1. An attention connection from query i to key j is causally valid iff **j ≤ i** (a position may attend only to itself and earlier positions).
> 2. For i = 2: allowed keys are **j ∈ {0, 1, 2}** (positions 0, 1, and itself; position 3 is in the future and is blocked).
> 3. Without the mask, position i can attend to **future tokens**, including the very next token it is supposed to predict → the answer leaks into the input. Training loss looks **very low** because the model just copies the visible future. But at inference the future does **not** exist (tokens are generated one at a time), so the model — having learned to rely on leaked future — produces **broken/incoherent** text.
> 4. softmax(x) = e^x / Σ e^x. Setting a masked logit to −∞ makes e^(−∞) = **0**, so that key gets **zero attention weight** after softmax → it contributes nothing, effectively blocking the connection.
> 5. Training is done in **parallel over all positions at once**, so without a causal mask each position would see its future tokens; the mask preserves the same left-to-right (autoregressive) constraint that inference must obey.

中文解析

- 关系：连接有效 ⇔ `j ≤ i`（只能看自己和更早）。
- i=2 允许 **j∈{0,1,2}**（位置 3 是未来，屏蔽）。
- 省略掩码→能看**未来 token**（含要预测的下一个）→答案泄漏；训练损失**异常低**（直接抄未来），但推理时未来不存在→模型崩坏、生成乱。
- −∞ 原因：softmax 里 e^(−∞)=**0** → 该键注意力权重为 0 → 不贡献 → 有效屏蔽。
- 训练并行处理所有位置，若无掩码每个位置会看到未来；掩码保持与推理一致的**从左到右**自回归约束。

Extra（201/202/203）速记

考点	必记
QKV 公式	`Attention(Q,K,V)=softmax(QKᵀ/√d_k)·V`

考点	必记
self vs cross	自注意力 Q,K,V 同源；交叉注意力 **Q←解码器、K,V←编码器输出**
LLM 架构	ChatGPT/DeepSeek = **decoder-only**，无交叉注意力
位置编码	`PE_{i,2k}=sin(i/10000^{2k/d})`、`PE_{i,2k+1}=cos(...)`；i=0→sin=0/cos=1；注入 `z_i=x_i+PE_i`
因果掩码	有效 ⇔ **j≤i**；i=2→{0,1,2}；masked 设 **−∞**（softmax 后=0）；防未来泄漏